VULKAN

Explicit GPU programming from first principles

UNIT 3

Synchronisation

Barriers, semaphores, fences, queues, frames in flight and render graphs, and how to prove them right

3.1 Why synchronisation is your job

3.2 The dependency model

3.3 Pipeline barriers in practice

3.4 Fences, semaphores and events

3.5 Queues and async compute

3.6 Frames in flight

3.7 Finding synchronisation bugs

3.8 Render graphs

First edition, October 2026

Contents

About this unit 2
Before you start 2
What this unit needs 2
3.1 Why synchronisation is your job 3
Core ideas 3
Reading 7
Labs 7
Problem set 8
3.2 The dependency model 10
Core ideas 10
Reading 16
Labs 16
Problem set 17
3.3 Pipeline barriers in practice 19
Core ideas 19
Reading 24
Labs 24
Problem set 26
3.4 Fences, semaphores and events 27
Core ideas 27
Reading 32
Labs 32
Problem set 33
3.5 Queues and async compute 35
Core ideas 35
Reading 39
Labs 39
Problem set 40
3.6 Frames in flight 42
Core ideas 42
Reading 46
Labs 46
Problem set 47
3.7 Finding synchronisation bugs 48
Core ideas 48
Reading 53
Labs 54
Problem set 55
3.8 Render graphs 56
Core ideas 56
Reading 68
Labs 68
Problem set 69
Sign-off 70

Vulkan, First edition, October 2026. © 2026 O'Neill Group LLC. Vulkan and the Vulkan logo are registered trademarks of the Khronos Group Inc. OpenGL and the oval logo are trademarks or registered trademarks of Hewlett Packard Enterprise in the United States and other countries, used by permission by Khronos. All other trademarks belong to their owners. This course is not endorsed by the Khronos Group.

UNIT 3

About this unit

Unit 3 teaches synchronisation, the part of Vulkan where most programs go wrong, so that you can apply it with confidence. It starts by showing what the GPU does when nothing constrains it, builds the dependency model that every barrier, semaphore and fence is made of, and turns it into patterns for real command streams. It then covers the primitives between the host and the device and between queues, keeps several frames in flight safely, and catalogues the classic bugs with the tools that find them. It ends with a render graph that derives every barrier from declarations of what each pass reads and writes.

Chapter	You build	Hours
3.1	A catalogue of nine hazards, each checked with and without its barrier	4
3.2	Barriers derived from the model for a table of scenarios, chains and layouts	6
3.3	A seven-pass pipeline with minimal barriers; the cost of barriers measured	6
3.4	A host and device pipeline on a timeline semaphore; split barriers with events	5
3.5	Uploads with ownership transfers; async compute overlap measured	6
3.6	A simulation loop with one, two and three frames in flight	5
3.7	Six classic synchronisation bugs, found and fixed	6
3.8	A render graph with barrier derivation, memory aliasing and queue scheduling	7

Before you start

Finish Units 1 and 2. Keep the Vulkan specification's synchronisation chapter open throughout this unit.

What this unit needs

The same machine and tools as Unit 2. A GPU with a separate compute or transfer queue makes Chapter 3.5's measurements more instructive.

CHAPTER 3.1

Why synchronisation is your job

TIME	4 hours
YOU BUILD	A catalogue of nine producer and consumer scenarios, each protected by its barrier and checked; then each run with its barrier removed, to see what synchronisation validation reports and whether the results go wrong.
NEEDS	Unit 1, and Chapters 2.1 and 2.2.

WHY IT MATTERS

Units 1 and 2 placed a barrier wherever one was needed and explained it only briefly. This unit teaches you to write barriers yourself, correctly and with no more of them than the work needs. This chapter shows why they are needed at all: what a GPU does with your commands when nothing constrains it, what OpenGL's driver used to do in your place, and the three kinds of hazard that every barrier exists to prevent. The rest of the unit builds the model (Chapter 3.2), the patterns (3.3), the other primitives (3.4 to 3.6), the tools (3.7) and finally a render graph that writes the barriers for you (3.8).

SKIP TEST

1. Commands A and B are recorded in that order in one command buffer, with nothing between them. Does Vulkan guarantee that A finishes before B starts? That B sees what A wrote?
2. Name the three kinds of hazard, and say which of them needs only an execution dependency.
3. Why can a read go wrong after a write that has already finished?
4. The CPU writes a mapped buffer after calling `vkQueueSubmit2` for a dispatch that reads it. What may the dispatch see?

If all four are easy, read the section on synchronisation validation and do Lab 3.1.3.

Core ideas

The device timeline

Chapter 1.3 introduced the device's timeline: the queue runs submitted work later than, and independently of, the host. Commands recorded into a command buffer, command buffers in a batch, and batches submitted to one queue are in *submission order*, which Vulkan uses to define what each barrier applies to. Submission order does not make one command wait for another, and says nothing about when commands finish. A GPU starts the next command as soon as it can, so that compute units freed by one dispatch's last workgroups can begin the next dispatch's first ones, and two commands with nothing between them can overlap and finish in either order (Figure 3.1.1).

Ordering is not enough on its own, because a write by a shader goes first into a cache, often one private to the compute unit that made it. Another compute unit that reads the same address may find an older value in its own cache, or in memory, even after the write has finished. For one command to see another's writes, those writes must be made *available*, written back from the caches that hold them to where others can reach them, and then *visible* to the reader, with any stale copies in the reader's caches discarded. A pipeline barrier does both, as well as ordering the commands. Chapter 3.2 makes these ideas precise.



Figure 3.1.1. Two dispatches on four compute units. Without a barrier, B's workgroups start on each unit as soon as A's leave it, while A is still running elsewhere. With a barrier, B waits until A has finished everywhere and A's writes have been made visible.

What OpenGL's driver did

An OpenGL driver tracks the last command that wrote each buffer and texture. When a later command reads one, the driver inserts the waits and the cache operations itself, usually conservatively. The exception is memory written by shaders, whose addresses the driver cannot know in advance: since OpenGL 4.2, a program must call `glMemoryBarrier` with bits that name how the data will be read next, such as `GL_SHADER_STORAGE_BARRIER_BIT` or `GL_COMMAND_BARRIER_BIT`.

That tracking costs CPU time on every call, synchronises more than the work needs because the driver must assume the worst, and cannot express what the program knows, such as that two pieces of work are independent and may overlap. A Vulkan driver does no such tracking, and every ordering between commands is one you record.

Three kinds of hazard

A *hazard* is a pair of accesses to the same memory whose result depends on their order. There are three kinds (Figure 3.1.2). A *read after write* (RAW) needs the write to finish, and its result to be made visible, before the read. A *write after read* (WAR) needs the read to finish before the write overwrites what it reads. Nothing needs to be made visible, so an execution dependency alone suffices. A *write after write* (WAW) needs the second write to land last, and the first write's data made available before it, so that a late write-back cannot overwrite the second. Two reads never conflict.

A catalogue of hazards

`u3_hazards` runs nine producer and consumer scenarios, one per common situation, and checks every result on the CPU. Each records its barrier with the raw synchronization2 structures. The first is the case Unit 2 met in every multi-pass kernel: one dispatch writes a buffer and the next reads it.

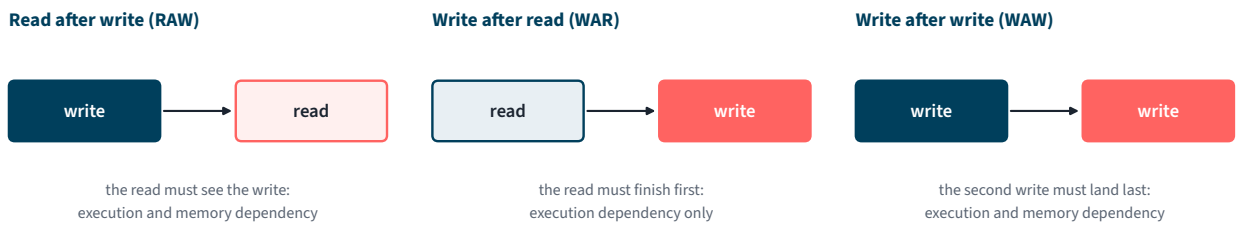


Figure 3.1.2. The three hazards. Only the write after read can be prevented by ordering alone.

Read after write between two dispatches code/u3/hazards/main.cpp

```
const VkMemoryBarrier2 barrier{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
};
const VkDependencyInfo dependency{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 1,
    .pMemoryBarriers = &barrier,
};
if (!bug) vkCmdPipelineBarrier2(cmd, &dependency);
```

A `VkMemoryBarrier2` names two *synchronisation scopes*. The source scope, `srcStageMask` and `srcAccessMask`, describes the earlier work and the writes that must be made available: compute shaders, and their writes to storage. The destination scope, `dstStageMask` and `dstAccessMask`, describes the later work that must wait and the accesses that must see the writes: compute shaders, and their reads of storage. `VkDependencyInfo` collects any number of such barriers, and `vkCmdPipelineBarrier2` records them. Chapter 3.2 explains every field.

The write after read needs only the ordering. Its barrier names the stages and leaves both access masks empty:

Write after read: an execution dependency, with no memory dependency code/u3/hazards/main.cpp

```
const VkMemoryBarrier2 barrier{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_NONE,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_NONE,
};
const VkDependencyInfo dependency{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 1,
    .pMemoryBarriers = &barrier,
};
if (!bug) vkCmdPipelineBarrier2(cmd, &dependency);
```

All nine scenarios, each with its barrier, checked on the M2 Pro

```
$ ./build/bin/u3_hazards
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
compute-compute  RAW  dispatch writes, dispatch reads           ok
war              WAR  dispatch reads, dispatch writes           ok
waw              WAW  dispatch writes, dispatch writes          ok
transfer-compute RAW  copy writes, dispatch reads               ok
compute-transfer RAW  dispatch writes, copy reads               ok
compute-indirect RAW  dispatch writes, indirect dispatch reads  ok
host-device      RAW  host writes, dispatch reads               ok
device-host      RAW  dispatch writes, host reads               ok
image-layout     RAW  dispatch writes image, copy reads         ok
PASS 9 cases
```

Removing the barriers

With `--bug`, each scenario leaves its barrier out, or in two cases breaks the synchronisation another way, and the program reports what synchronisation validation said and whether the results were wrong. The first case shows the usual outcome:

The read after write without its barrier

```
$ ./build/bin/u3_hazards --case compute-compute --bug
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
[vulkan error] vkCmdDispatch(): READ_AFTER_WRITE hazard detected. Shader stage
    VK_SHADER_STAGE_COMPUTE_BIT reads VkBuffer 0x...[a], which was previously written by
    another vkCmdDispatch command.
The buffer is referenced by descriptor binding 0 (VK_DESCRIPTOR_TYPE_STORAGE_BUFFER) from
    descriptor set 0 (VkDescriptorSet 0x...), VkPipeline 0x...[transform.comp].
No sufficient synchronization is present to ensure that a read
    (VK_ACCESS_2_SHADER_STORAGE_READ_BIT) at VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT does not
    conflict with a prior write (VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT) at the same stage.
compute-compute  RAW  dispatch writes, dispatch reads          bug: no barrier
  validation: SYNC-HAZARD-READ-AFTER-WRITE
  results: correct
error: --bug: 1 detected, 0 undetected
vkf: the validation layers reported 1 error(s)
```

Table 3.1.1 summarises all nine. Synchronisation validation found every hazard between two commands, and the results were correct in every one of those cases on the M2 Pro, because its GPU happened to finish the first command before starting the second. The validation layer does not track the host's accesses, so it reported neither of the two hazards between the host and the device, and one of them produced wrong results.

Case	Hazard	Validation reported	Results
`compute-compute`	RAW	`SYNC-HAZARD-READ-AFTER-WRITE`	correct
`war`	WAR	`SYNC-HAZARD-WRITE-AFTER-READ`	correct
`waw`	WAW	`SYNC-HAZARD-WRITE-AFTER-WRITE`	correct
`transfer-compute`	RAW	`SYNC-HAZARD-READ-AFTER-WRITE`	correct
`compute-transfer`	RAW	`SYNC-HAZARD-READ-AFTER-WRITE`	correct
`compute-indirect`	RAW	`SYNC-HAZARD-READ-AFTER-WRITE`	correct
`host-device`	RAW	nothing: host accesses are not tracked	1,029,928 values wrong
`device-host`	RAW	nothing: host accesses are not tracked	correct
`image-layout`	RAW	`SYNC-HAZARD-WRITE-AFTER-WRITE`, against the layout transition	correct

Table 3.1.1. The nine scenarios with their synchronisation removed or broken, on the M2 Pro

The case that went wrong writes on the host. The correct version finishes writing a mapped buffer before submitting the dispatch that reads it, because host writes made before `vkQueueSubmit2` are visible to the submitted work. The broken version submits first and writes afterwards, while the GPU is already reading:

Host writes before and after the submission that reads them — code/u3/hazards/main.cpp

```
VkCommandBuffer cmd = vkf::allocateCommandBuffer(e.ctx, e.ctx.commandPool());
vkf::beginCommands(cmd);
e.transformBuffer(cmd, e.upload, e.uploadOut, HostCount);
vkf::endCommands(cmd);
const auto fence = vkf::createFence(e.ctx);
if (!bug) writeHostValues(e.upload, 10);
vkf::submit(e.ctx.mainQueue(), std::span(&cmd, 1), {}, {}, fence);
if (bug) writeHostValues(e.upload, 10);
vkf::waitFence(e.ctx, fence);
```

Writing after the submission: the GPU read the buffer while the CPU was still writing it

```
$ ./build/bin/u3_hazards --case host-device --bug
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
host-device      RAW  host writes, dispatch reads              bug: write after submit
  validation: nothing (host accesses are not tracked)
  results: 1029928 values wrong
error: --bug: 1 detected, 0 undetected
```

The GPU read a mixture of old and new values, and almost every result was wrong. No layer can catch this, because the bug is in what the CPU does between two Vulkan calls.

> KEY IDEA
> Without a barrier, two commands are unordered, and a write need not be visible to any other command. A barrier orders the commands on either side of one point in the command stream and makes the earlier writes visible to the later ones. A correct result without it shows only that the timing happened to work on that run.

Synchronisation validation

Synchronisation validation records, for every range of every buffer and image, the last accesses made to it, with their stages and access types, and the barriers recorded since. When a new access arrives, it checks whether those barriers order it after every conflicting earlier access and make the earlier writes visible to it. If not, it reports the hazard, the two commands involved, and the stages and accesses that a barrier would need to connect, which is usually enough to write the barrier. It also checks the boundaries between command buffers and between submissions, including semaphores.

It has three limits that this chapter has shown or that matter later. It does not see host accesses, so the rules for mapped memory are yours to keep. It does not see races inside a single dispatch, such as those of Chapter 2.2. Finally, it learns which buffers a shader reads and writes from a static analysis of the SPIR-V, the heuristic that the course's code enables, and so can occasionally report a hazard that cannot happen. Every program in this unit runs with synchronisation validation on and must be silent.

> COMING FROM OPENGL
> In OpenGL, the two dispatches of the first case need `glMemoryBarrier(GL_SHADER_STORAGE_BARRIER_BIT)` between them, the indirect dispatch needs `GL_COMMAND_BARRIER_BIT`, and a buffer read back by `glGetBufferSubData` after a shader wrote it needs `GL_BUFFER_UPDATE_BARRIER_BIT`. Copies need nothing, because the driver tracks them, and neither do the host cases, because `glBufferSubData` and `glGetBufferSubData` wait for the GPU when they must. Every one of those implicit waits is a barrier, or a stall, that Vulkan leaves to you.

Reading

- The Vulkan specification, chapter "Synchronization and Cache Control", the introduction and the section "Execution and Memory Dependencies".
- Hans-Kristian Arntzen, "Yet another blog explaining Vulkan synchronization" (2019), the sections up to pipeline barriers.
- The validation layer's documentation on synchronisation validation, in the Vulkan-ValidationLayers repository.

Labs

LAB 3.1.1 Run the catalogue — 1 hour

Goal. Record which hazards validation reports on your GPU.

Code. `code/u3/hazards`

1. Run `build/bin/u3_hazards` and confirm that all nine cases pass.
2. Run each case with `--case NAME --bug`, and build your own version of Table 3.1.1.
3. For one RAW case, one WAR case and the image case, read the full validation message and identify the stages and accesses it says a barrier must connect.

DONE WHEN

- [ ] Your table has nine rows, with what was reported and whether the results were correct.
- [ ] For three cases, the barrier you would write from the message matches the one in the code.

EVIDENCE FOR THE NOTEBOOK

- The table and the three messages, annotated.

LAB 3.1.2 Try to make a hazard show — 1.5 hours

Goal. Find out whether changes in timing make a missing barrier give wrong results.

Code. `code/u3/hazards`

1. In a copy of the compute-compute case, make the producer slower, for example by repeating its work in a loop, and the consumer faster, and run it with the barrier removed.
2. Try at least three variations of size and work, with validation on and off, and record whether the results go wrong.
3. Run the host-device case with `--bug` several times and record how many values are wrong each time.

DONE WHEN

- [ ] You have recorded the outcome of every variation.
- [ ] You can explain why your GPU did or did not produce wrong results, and why that tells you nothing about other GPUs.

EVIDENCE FOR THE NOTEBOOK

- The variations, their outcomes and your explanation.

LAB 3.1.3 Write the barriers yourself — 1.5 hours

Goal. Place every barrier in a chain of commands.

Code. `code/u3/hazards`

1. Add a case that runs this chain: fill buffer `a` with a dispatch; copy `a` to `b`; transform `b` into `a` with a dispatch; copy `a` to the readback buffer; read it on the host.
2. Write the barriers it needs with raw `VkMemoryBarrier2` structures, and check the result.
3. Remove each barrier in turn, predict what synchronisation validation will report, and run.

DONE WHEN

- [ ] The chain runs correctly with validation silent.
- [ ] Each of your predictions names the hazard type and the two commands, and matches the report.

EVIDENCE FOR THE NOTEBOOK

- The barriers with a sentence each, and the predictions next to the reports.

Problem set

1. Give an example of each kind of hazard from the programs of Units 1 and 2.
2. Why does a write after read need no access masks? What would adding them cost, if anything?
3. A barrier between a dispatch that writes a buffer and one that reads it has both stage masks set to `VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT` and both access masks empty. Is the program correct? What will synchronisation validation report?

4. Explain why, on the M2 Pro, the host-device bug produced wrong results while every missing barrier between commands did not.
5. For each of the nine scenarios, give the OpenGL call, if any, that would order it, and say what the OpenGL driver does when no call is needed.

COMPLETION CHECKLIST

- ☐ I can explain what submission order guarantees and what it does not.
- ☐ I can explain availability and visibility, and why finished writes may still be invisible.
- ☐ I can name the three hazards, and say which needs a memory dependency.
- ☐ I can read a synchronisation validation message and write the barrier it asks for, and I know what the layer cannot see.
- ☐ Lab 3.1.1–3.1.3 done-when criteria all hold, with evidence filed.

CHAPTER 3.2

The dependency model

TIME	6 hours
YOU BUILD	Barriers for a table of producer and consumer scenarios, each derived from the model and checked by synchronisation validation; the same barriers made too narrow and too wide; a dependency split into a chain; and image layout transitions placed by hand.
NEEDS	Chapter 3.1.

WHY IT MATTERS

Chapter 3.1 showed that barriers are necessary. This chapter shows what a barrier means, precisely, so that you can write the right one for any situation instead of copying one that seems similar. The model has few parts: two synchronisation scopes, the pipeline stages that bound them, the access scopes that say which memory accesses are covered, and the availability and visibility operations that move data between them. Everything else in this unit, from semaphores to render graphs, is built from the same parts.

SKIP TEST

1. What is the difference between an execution dependency and a memory dependency, and when is the first enough?
2. A barrier's `srcStageMask` is `VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT`. Does it wait for a copy recorded before it?
3. Why do access masks apply only to the stages named in the stage masks, while stage masks also cover logically earlier or later stages?
4. What does a layout transition do to an image's memory, and where in a barrier does it happen?
5. Can one dependency be split across two barriers? Under what condition?

If all five are easy, read the section on chains and do Labs 3.2.1 and 3.2.3.

Core ideas

Execution dependencies

An *execution dependency* between two sets of operations guarantees that every operation in the first set happens before every operation in the second. A pipeline barrier creates one between the commands recorded before it and the commands recorded after it, but only for the parts of those commands that its stage masks name. Its *first synchronisation scope* is the work of the stages in `srcStageMask` in the commands before the barrier, and its *second synchronisation scope* is the work of the stages in `dstStageMask` in the commands after it. Work outside the scopes is not ordered at all.

The stages come from the list in Figure 3.2.2. Each command runs some of them: a dispatch runs `DRAW_INDIRECT`, if indirect, and `COMPUTE_SHADER`; a copy runs `COPY`; a draw runs the graphics stages. A stage mask also covers the stages that come logically before the ones it names, in a source mask, or logically after them, in a destination mask, in the order the figure shows for each kind of command. A source mask of `COLOR_ATTACHMENT_OUTPUT` therefore waits for the vertex and fragment shaders too.

Three stage values are special. `VK_PIPELINE_STAGE_2_NONE` names no stage: as a source it waits for nothing, and as a destination it blocks nothing. `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT` names every stage of every

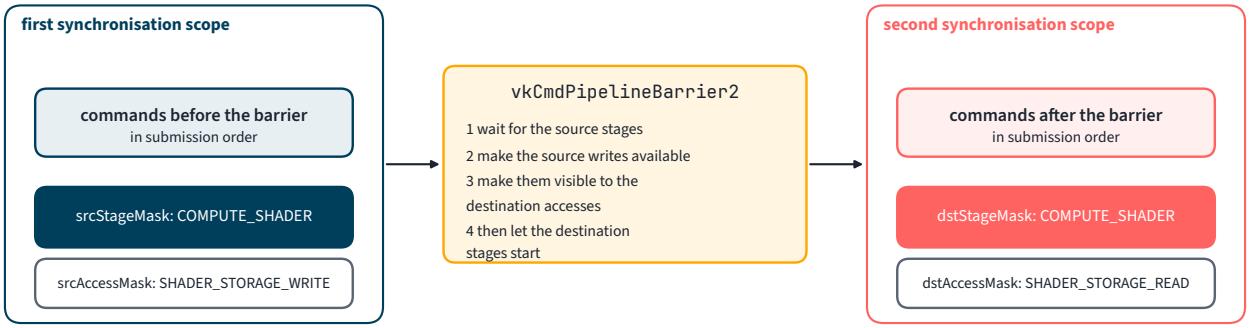


Figure 3.2.1. What a pipeline barrier does. Work in the first scope finishes and its writes in the first access scope are made available and then visible to the second access scope, before work in the second scope starts.

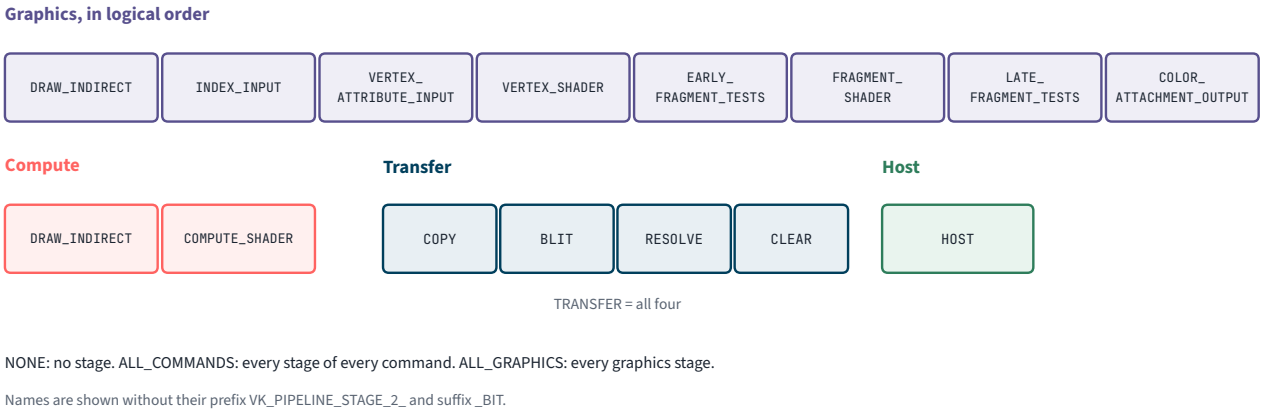


Figure 3.2.2. The synchronization2 pipeline stages this course uses, in logical order for each kind of command. Tessellation and geometry shaders, which the course does not use, come between the vertex shader and the early fragment tests.

command, and is correct everywhere but often slower than necessary. `VK_PIPELINE_STAGE_2_HOST_BIT` is the host's reads and writes of mapped memory, which a barrier can only make writes available to or visible from. The host itself never waits on a barrier.

Memory dependencies

An execution dependency is not enough when data passes between the two sides, because of caches. A *memory dependency* adds three operations, which happen in order between the two scopes:

1. An *availability operation* makes the writes in the first *access scope* available: written back from whatever caches hold them to a level where every later access in the same *memory domain* can find them.
2. A *memory domain operation*, when the barrier names the host as a destination, moves available writes from the device's domain to the host's.
3. A *visibility operation* makes available writes visible to the accesses in the second access scope: stale copies in the caches those accesses use are discarded, so that they read the new values.

The access scopes come from the access masks: `srcAccessMask` names the kinds of write to make available, and `dstAccessMask` the kinds of access to make them visible to. Unlike stage masks, access masks apply only to accesses performed by the stages named explicitly, never to logically earlier or later ones, and each access flag is valid only with the stages that can perform it. The flags this unit uses most are `SHADER_STORAGE_READ` and `SHADER_STORAGE_WRITE` for storage buffers and images, `SHADER_SAMPLED_READ` for sampled images, `UNIFORM_READ`, `TRANSFER_READ` and `TRANSFER_WRITE` for copies, `INDIRECT_COMMAND_READ` for indirect parameters, `HOST_READ` and `HOST_WRITE`, and the catch-alls `MEMORY_READ` and `MEMORY_WRITE`.

> **KEY IDEA**
> Read a barrier as one sentence: *wait for these stages of earlier commands to finish, make these of their writes available, make them visible to these accesses of later commands, then let these stages of later commands begin.* Every field of `VkMemoryBarrier2` is one phrase of that sentence.

A write after read needs only the execution dependency, because the read has nothing to make visible; Chapter 3.1's `war` case leaves both access masks empty for this reason. A read after write and a write after write need both.

Three kinds of barrier

`VkDependencyInfo` carries three arrays of barriers. A *global* barrier, `VkMemoryBarrier2`, covers every resource. A *buffer* barrier, `VkBufferMemoryBarrier2`, adds a buffer and a range, and an *image* barrier, `VkImageMemoryBarrier2`, adds an image, a subresource range and a pair of layouts. Both of the latter can also transfer a resource between queue families, which Chapter 3.5 explains. Chapter 3.1's copy-then-dispatch case is written with a buffer barrier:

A buffer barrier from a copy's writes to a dispatch's reads — code/u3/hazards/main.cpp

```
const VkBufferMemoryBarrier2 barrier{
    .sType = VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
    .srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
    .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .buffer = e.a,
    .offset = 0,
    .size = VK_WHOLE_SIZE,
};
const VkDependencyInfo dependency{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .bufferMemoryBarrierCount = 1,
    .pBufferMemoryBarriers = &barrier,
};
if (!bug) vkCmdPipelineBarrier2(cmd, &dependency);
```

On most current GPUs a buffer barrier costs the same as a global barrier with the same masks, because caches are not flushed one buffer at a time. Use global barriers for buffers on a single queue: they are shorter to write and cannot leave out a buffer. Use buffer and image barriers when they carry something a global barrier cannot: a layout transition, or a transfer of ownership.

Barriers to and from the host

The host is a stage of its own. When a dispatch writes results the CPU will read, the barrier's destination is the host, which adds the memory domain operation to the host's domain, and the CPU then waits on a fence before reading:

Making shader writes available to the host — code/u3/hazards/main.cpp

```
const VkMemoryBarrier2 barrier{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_HOST_BIT,
    .dstAccessMask = VK_ACCESS_2_HOST_READ_BIT,
};
const VkDependencyInfo dependency{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 1,
    .pMemoryBarriers = &barrier,
};
if (!bug) vkCmdPipelineBarrier2(cmd, &dependency);
```

If the memory is not `HOST_COHERENT`, the CPU must also invalidate the range before reading it, as Chapter 1.2 described. `vkf::Buffer::invalidate` does nothing when the memory is coherent.

Invalidating before the host reads, from Chapter 3.7's catalogue of bugs — code/u3/bugs/main.cpp

```
// Without an invalidate, the host may read stale lines from its own caches.
if (!bug) {
    const VkMappedMemoryRange result{
        .sType = VK_STRUCTURE_TYPE_MAPPED_MEMORY_RANGE,
        .memory = output.memory,
        .offset = 0,
        .size = VK_WHOLE_SIZE,
    };
    VKF_CHECK(vkInvalidateMappedMemoryRanges(e.ctx.device(), 1, &result));
}
```

In the other direction no barrier is needed, because submitting work makes the host's earlier writes visible to it, provided they were flushed when the memory is not coherent. The one constraint, which Chapter 3.1's `host-device` case broke, is that the host must not write memory that submitted work may still be reading.

When a copy and the host both follow a dispatch, each needs its own destination. Chapter 3.1's `compute-transfer` case orders the copy after the dispatch, then the host after the copy:

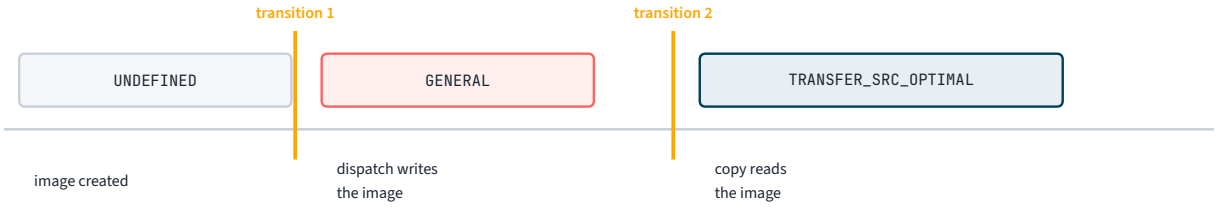
Shader writes, a copy, and a read on the host — code/u3/hazards/main.cpp

```
const VkMemoryBarrier2 toCopy{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
    .dstAccessMask = VK_ACCESS_2_TRANSFER_READ_BIT,
};
const VkDependencyInfo beforeCopy{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 1,
    .pMemoryBarriers = &toCopy,
};
if (!bug) vkCmdPipelineBarrier2(cmd, &beforeCopy);
const VkBufferCopy region{.size = Count * 4};
vkCmdCopyBuffer(cmd, e.a, e.readback, 1, &region);
const VkMemoryBarrier2 toHost{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
    .srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_HOST_BIT,
    .dstAccessMask = VK_ACCESS_2_HOST_READ_BIT,
};
const VkDependencyInfo afterCopy{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 1,
    .pMemoryBarriers = &toHost,
};
vkCmdPipelineBarrier2(cmd, &afterCopy);
```

Image layouts and their transitions

An image's *layout* is the arrangement of its data that the hardware uses for a kind of access, as Chapter 1.2 introduced. A program declares every change of layout in an image barrier, with `oldLayout` and `newLayout`, and the device performs a *layout transition*: it rearranges, compresses or decompresses the image's data as needed. A transition reads and writes the whole subresource range, so it is ordered like a write: after the barrier's first scope and its availability operation, and before its visibility operation and second scope (Figure 3.2.3).

An `oldLayout` of `VK_IMAGE_LAYOUT_UNDEFINED` promises that the old contents do not matter, so the transition need not preserve them, and an image that nothing has used yet needs nothing in the first scope:



Each transition is part of a barrier: it happens after the barrier's source scope and before its destination scope, and it both reads and writes the image.

Figure 3.2.3. The image in Chapter 3.1's `image-layout` case. Each transition belongs to a barrier and sits between that barrier's two scopes.

From UNDEFINED to GENERAL before a shader writes the image — code/u3/hazards/main.cpp

```
const VkImageMemoryBarrier2 toGeneral{
    .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_NONE,
    .srcAccessMask = VK_ACCESS_2_NONE,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .oldLayout = VK_IMAGE_LAYOUT_UNDEFINED,
    .newLayout = VK_IMAGE_LAYOUT_GENERAL,
    .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .image = e.image,
    .subresourceRange = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1},
};
const VkDependencyInfo beforeWrite{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .imageMemoryBarrierCount = 1,
    .pImageMemoryBarriers = &toGeneral,
};
vkCmdPipelineBarrier2(cmd, &beforeWrite);
u3::dispatch(cmd, e.pattern, set, SeedPush{12}, Side / 16, Side / 16);
```

The second transition must wait for the shader's writes, because it rewrites the image itself. Chapter 3.1's broken version of this barrier left the source scope empty, and synchronisation validation reported a write after write between the shader's writes and the transition.

From GENERAL to TRANSFER_SRC_OPTIMAL before the copy reads it — code/u3/hazards/main.cpp

```
VkImageMemoryBarrier2 toTransfer{
    .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
    .dstAccessMask = VK_ACCESS_2_TRANSFER_READ_BIT,
    .oldLayout = VK_IMAGE_LAYOUT_GENERAL,
    .newLayout = VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
    .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .image = e.image,
    .subresourceRange = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1},
};
if (bug) {
    toTransfer.srcStageMask = VK_PIPELINE_STAGE_2_NONE;
    toTransfer.srcAccessMask = VK_ACCESS_2_NONE;
}
const VkDependencyInfo beforeCopy{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .imageMemoryBarrierCount = 1,
    .pImageMemoryBarriers = &toTransfer,
};
vkCmdPipelineBarrier2(cmd, &beforeCopy);
```

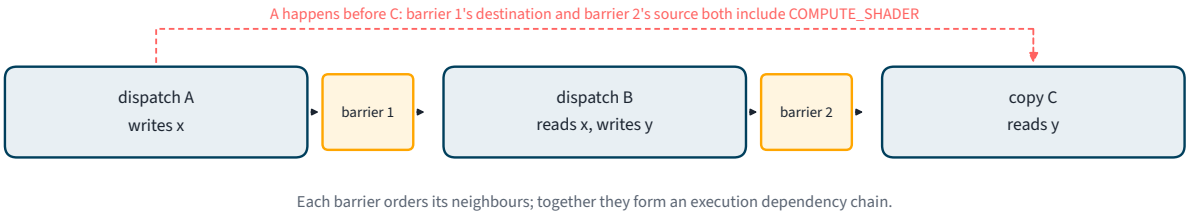


Figure 3.2.4. An execution dependency chain. A happens before B by barrier 1 and B before C by barrier 2; because both barriers include the compute stage between them, A also happens before C.

> **PITFALL**
> `oldLayout` must be the layout the image is actually in, or `VK_IMAGE_LAYOUT_UNDEFINED`. Vulkan does not track layouts for you. Claiming the wrong old layout is undefined behaviour, which the validation layer reports when it can see it. Using `UNDEFINED` on an image whose contents you still need discards them.

Chains

If a barrier's second synchronisation scope and a later barrier's first scope share a stage, the two barriers form an *execution dependency chain*: the first barrier's first scope happens before the second barrier's second scope, as if a single barrier had joined them (Figure 3.2.4). The same holds for memory: a write made available by the first barrier can be made visible by the second.

Chains are why a barrier between every pair of consecutive passes is enough in a long pipeline, and why a barrier need not name stages far back in the command stream. They also explain why a barrier whose source stages do not include the stage of the command you meant to wait for does not wait for that command, however close the two are in the command buffer.

Writing a barrier

The model reduces barrier writing to a procedure:

1. Find each resource's last writer before the point, and the stage and access of the write.
2. Find each access after the point that conflicts with it, and its stage and access.
3. For images, find the layout the next access needs.
4. Write one barrier per hazard: global for buffers, image barriers for images, with exactly those stages and accesses.
5. Put all the barriers at one point into one `VkDependencyInfo`, so that the device synchronises once.

Table 3.2.1 lists the combinations this unit meets most; Chapter 3.3 gives the full catalogue.

Producer	Consumer	Source → destination
dispatch writes a buffer	dispatch reads it	`COMPUTE_SHADER`, `SHADER_STORAGE_WRITE` → `COMPUTE_SHADER`, `SHADER_STORAGE_READ`
dispatch reads a buffer	dispatch overwrites it	`COMPUTE_SHADER`, none → `COMPUTE_SHADER`, none
copy or fill writes	dispatch reads	`COPY` or `CLEAR`, `TRANSFER_WRITE` → `COMPUTE_SHADER`, `SHADER_STORAGE_READ`
dispatch writes	copy reads	`COMPUTE_SHADER`, `SHADER_STORAGE_WRITE` → `COPY`, `TRANSFER_READ`
dispatch writes parameters	indirect dispatch or draw	`COMPUTE_SHADER`, `SHADER_STORAGE_WRITE` → `DRAW_INDIRECT`, `INDIRECT_COMMAND_READ`
any device write	host reads after a fence	the writer's stage and access → `HOST`, `HOST_READ`
nothing yet	first write to an image	`NONE`, none → the writer's stage and access, with `UNDEFINED` → the layout

Table 3.2.1. Common dependencies, with the prefixes `VK_PIPELINE_STAGE_2_` and `VK_ACCESS_2_` and the suffix `_BIT` left out

COMING FROM OPENGL

`glMemoryBarrier`'s bits each name a kind of later access, such as `GL_SHADER_STORAGE_BARRIER_BIT` or `GL_COMMAND_BARRIER_BIT`, and correspond to Vulkan's destination access masks. OpenGL has no source scope: every earlier write of the relevant kind is made visible, from every stage. It has no destination stage either, and no layouts, which its driver manages.

Reading

- The Vulkan specification, chapter "Synchronization and Cache Control": "Execution and Memory Dependencies", "Pipeline Stages", "Access Types", "Pipeline Barriers" and "Image Layout Transitions". Read the definitions slowly: this chapter paraphrases them.
- The Vulkan Guide, "Synchronization" and "VK_KHR_synchronization2".
- Hans-Kristian Arntzen, "Yet another blog explaining Vulkan synchronization" (2019), in full.

Labs

LAB 3.2.1 Barriers for a table of scenarios — 2 hours

Goal. Derive barriers from the model and have them checked.

Code. `code/u3/hazards`

1. Add four cases to `u3_hazards`: `vkCmdFillBuffer` then a dispatch that reads; a dispatch that writes a buffer the next dispatch reads as a uniform buffer; a dispatch that writes an image the next dispatch samples in `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`; two copies into the same buffer.
2. For each, write the barrier from the procedure above before running anything, with the hazard type in a comment in your notebook.
3. Run each with validation on, then with its barrier removed.

DONE WHEN

- [ ] All four cases pass with synchronisation validation silent.
- [ ] With each barrier removed, the reported hazard is the one you predicted.

EVIDENCE FOR THE NOTEBOOK

- The four barriers with your derivations, and the reports.

LAB 3.2.2 Too narrow and too wide — 1 hour

Goal. Learn which mistakes validation catches and which it cannot.

Code. `code/u3/hazards`

1. In the `compute-compute` case, try these destinations in turn: `TRANSFER_READ` instead of `SHADER_STORAGE_READ`; the `FRAGMENT_SHADER` stage instead of `COMPUTE_SHADER`; and `ALL_COMMANDS` with `MEMORY_READ`.
2. Try a source of `ALL_COMMANDS` with `MEMORY_WRITE`.
3. Classify each as wrong, correct but wider than needed, or correct and minimal.

DONE WHEN

- [ ] You have the validation output for each variant.
- [ ] Your classification agrees with the model, and you can say why validation accepts the wide versions.

EVIDENCE FOR THE NOTEBOOK
- The variants, their outputs and the classification.

LAB 3.2.3 Split a dependency into a chain — 1.5 hours

Goal. Use the chain rule deliberately.

Code. `code/u3/hazards`

1. Between the two dispatches of `compute-compute`, record an unrelated copy, and replace the barrier with two: from the dispatch's writes to the `COPY` stage with no access, and from the `COPY` stage with no access to the compute shader's reads.
2. Run with validation on. Then change the second barrier's source stage to `HOST` and run again.

DONE WHEN
- [ ] You have recorded what synchronisation validation says about each version.
- [ ] You can explain each result with the definition of an execution dependency chain.

EVIDENCE FOR THE NOTEBOOK
- The two versions and the explanations.

LAB 3.2.4 Layouts by hand — 1.5 hours

Goal. Place layout transitions and see what goes wrong without them.

Code. `code/u3/hazards`

1. In the `image-layout` case, copy from the image in `VK_IMAGE_LAYOUT_GENERAL` without the second transition, adjusting the barrier's other fields, and confirm that it is valid.
2. Claim the wrong `oldLayout`, `TRANSFER_DST_OPTIMAL`, in the second barrier, and record the validation message.
3. Use `UNDEFINED` as the `oldLayout` of the second barrier and record what happens to the data.

DONE WHEN
- [ ] You have the three outcomes recorded.
- [ ] You can explain when staying in `GENERAL` is acceptable and when a specific layout is worth a transition.

EVIDENCE FOR THE NOTEBOOK
- The three outcomes and your explanation.

Problem set

1. Write the `VkMemoryBarrier2` for a dispatch that writes a buffer that the next dispatch reads as a uniform buffer, and for the reverse, where a dispatch overwrites a buffer that the previous one read as uniforms.
2. A barrier's `srcStageMask` is `VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT`. Does it wait for a copy recorded before it? Use Figure 3.2.2 to explain.
3. Explain availability and visibility using two compute units with private caches, one writing a value and one reading it.
4. Why is `VK_PIPELINE_STAGE_2_NONE` a correct source scope for an image's first transition from `UNDEFINED`? When would it be wrong for a transition from `UNDEFINED`?
5. Two dispatches write disjoint halves of a buffer, and a third reads all of it. Which barriers are needed, and where? Would two buffer barriers with ranges be better than one global barrier?

COMPLETION CHECKLIST

- ☐ I can state what an execution dependency and a memory dependency each guarantee.
- ☐ I can choose stage and access masks for any producer and consumer, and say what the stage expansion covers.
- ☐ I can place layout transitions and explain what they do to an image's memory.
- ☐ I can use chains, and write barriers with the five-step procedure.
- ☐ Lab 3.2.1–3.2.4 done-when criteria all hold, with evidence filed.

CHAPTER 3.3

Pipeline barriers in practice

TIME	6 hours
YOU BUILD	A seven-pass compute pipeline, from generating a field to an indirect dispatch and a read on the host, with the fewest barriers it can have, checked against the CPU; and an experiment that measures what barriers cost, precise and over-wide, between dependent and independent dispatches.
NEEDS	Chapter 3.2.

WHY IT MATTERS

Chapter 3.2 gave the model and a procedure for applying it. This chapter is about using it well in real command streams: a catalogue of the patterns that recur in compute and graphics work, a pipeline of several passes with exactly the barriers it needs and no others, and a measurement of what barriers cost. A barrier is the point where the GPU stops overlapping work, so every unnecessary barrier, and every barrier wider than it needs to be, can leave hardware idle.

SKIP TEST

1. A compute pass writes a vertex buffer that the next draw reads. Write the barrier's four masks.
2. Two independent dispatches each write a buffer that a third reads. How many barriers do you need, and where?
3. A barrier's source scope is the compute stage. Which earlier dispatches does it wait for: the one that wrote the data, or all of them?
4. Why might `VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT` with `MEMORY_WRITE` and `MEMORY_READ` be correct but slow?

If all four are easy, read the section on cost and do Labs 3.3.2 and 3.3.3.

Core ideas

A catalogue of dependencies

Table 3.3.1 extends Chapter 3.2's list with the graphics cases that Unit 4 meets. Each row is one hazard. A real program finds its rows, then batches the barriers that fall at the same point.

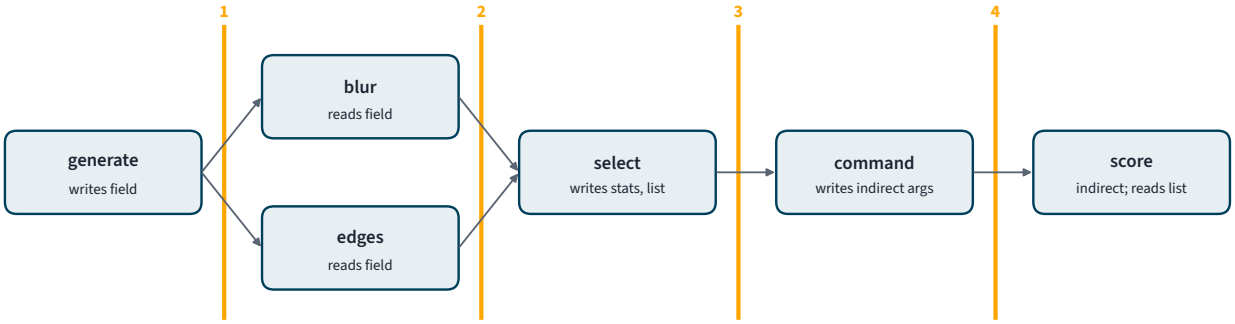
Producer	Consumer	Source → destination, and layouts
dispatch writes storage	dispatch reads or writes it	`COMPUTE_SHADER` , `SHADER_STORAGE_WRITE` → `COMPUTE_SHADER` , `SHADER_STORAGE_READ` (and `_WRITE`)
dispatch reads storage	dispatch writes it	`COMPUTE_SHADER` , none → `COMPUTE_SHADER` , none
copy, fill or update writes	dispatch reads	`COPY` , `CLEAR` or `TRANSFER` , `TRANSFER_WRITE` → `COMPUTE_SHADER` , `SHADER_STORAGE_READ`
dispatch writes	copy reads, for readback	`COMPUTE_SHADER` , `SHADER_STORAGE_WRITE` → `COPY` , `TRANSFER_READ`
dispatch writes parameters	indirect dispatch or draw	`COMPUTE_SHADER` , `SHADER_STORAGE_WRITE` → `DRAW_INDIRECT` , `INDIRECT_COMMAND_READ`
dispatch writes vertices	draw reads them	`COMPUTE_SHADER` , `SHADER_STORAGE_WRITE` → `VERTEX_ATTRIBUTE_INPUT` , `VERTEX_ATTRIBUTE_READ`
dispatch writes indices	indexed draw	`COMPUTE_SHADER` , `SHADER_STORAGE_WRITE` → `INDEX_INPUT` , `INDEX_READ`
dispatch writes an image	draw samples it	`COMPUTE_SHADER` , `SHADER_STORAGE_WRITE` → `FRAGMENT_SHADER` , `SHADER_SAMPLED_READ` ; `GENERAL` → `SHADER_READ_ONLY_OPTIMAL`
draw renders an image	dispatch reads it	`COLOR_ATTACHMENT_OUTPUT` , `COLOR_ATTACHMENT_WRITE` → `COMPUTE_SHADER` , `SHADER_SAMPLED_READ` ; `COLOR_ATTACHMENT_OPTIMAL` → `SHADER_READ_ONLY_OPTIMAL`
copy writes an image	draw samples it	`COPY` , `TRANSFER_WRITE` → `FRAGMENT_SHADER` , `SHADER_SAMPLED_READ` ; `TRANSFER_DST_OPTIMAL` → `SHADER_READ_ONLY_OPTIMAL`
device writes	host reads after a fence	the writer's stage and access → `HOST` , `HOST_READ` ; then invalidate if not coherent
host writes	submitted work reads	no barrier: write and flush before `vkQueueSubmit2`

Table 3.3.1. The dependencies this course uses, with prefixes and suffixes left out as in Chapter 3.2

A pipeline of seven passes

`u3_passes` runs a small image-analysis pipeline over a 512 × 512 field of values and checks its final statistics against the CPU (Figure 3.3.1). It generates the field into an image; computes a blur and an edge measure of it in two independent passes; selects the cells whose blur is high and whose edge measure is low, counting them with atomics and listing their indices; writes an indirect dispatch command sized to the number selected; scores the selected cells in that indirect dispatch; and copies the statistics to the host. It has seven barriers, one at each point where data passes between passes, and no more.

The first barrier serves two consumers. The blur and the edge passes both read the field and write different buffers, so neither needs a barrier against the other, and they may run at the same time:



1 the field's writes → shader reads
2 blur and edges' writes, and the clear of stats → shader reads and writes
3 select's writes → shader reads and writes
4 command's writes → DRAW_INDIRECT, INDIRECT_COMMAND_READ

Figure 3.3.1. The pipeline and its four barriers between passes. Before them, a transition takes the field's image from `UNDEFINED` to `GENERAL` ; after them, two barriers carry the statistics to a copy and then to the host.

One barrier for two independent consumers code/u3/passes/main.cpp

```
const VkMemoryBarrier2 fieldWritten{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
};
const VkDependencyInfo beforeFilters{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 1,
    .pMemoryBarriers = &fieldWritten,
};
vkCmdPipelineBarrier2(cmd, &beforeFilters);
u3::dispatch(cmd, blur, blurSet, tiles, tiles);
u3::dispatch(cmd, edges, edgesSet, tiles, tiles);
```

Before the selection, two dependencies meet: the filters' outputs, written by shaders, and the statistics buffer, cleared by `vkCmdFillBuffer` at the start of the command buffer and about to be updated atomically. One `VkDependencyInfo` carries both barriers, so the device synchronises once:

Two barriers batched into one call code/u3/passes/main.cpp

```
const VkMemoryBarrier2 beforeSelect[] = {
    {
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
        .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
    },
    {
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_CLEAR_BIT,
        .srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
        .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .dstAccessMask =
            VK_ACCESS_2_SHADER_STORAGE_READ_BIT | VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    },
};
const VkDependencyInfo selectDependency{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 2,
    .pMemoryBarriers = beforeSelect,
};
vkCmdPipelineBarrier2(cmd, &selectDependency);
const SelectPush thresholds{cells, MinBlurred, MaxEdge};
u3::dispatch(cmd, select, selectSet, thresholds, vkf::groupCount(cells, 256));
```

The command pass writes a `VkDispatchIndirectCommand`, which the indirect dispatch reads at the `DRAW_INDIRECT` stage, before any shader runs. A barrier that named only the compute stage as its destination would not cover that read:

Indirect parameters written by a shader code/u3/passes/main.cpp

```
const VkMemoryBarrier2 commandWritten{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT,
    .dstAccessMask = VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT,
};
const VkDependencyInfo beforeScore{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .memoryBarrierCount = 1,
    .pMemoryBarriers = &commandWritten,
};
vkCmdPipelineBarrier2(cmd, &beforeScore);
u3::bind(cmd, score, scoreSet);
vkCmdDispatchIndirect(cmd, commandBuffer, 0);
```

The pipeline, checked against the CPU, followed by the cost experiment, with validation on

```
$ ./build/bin/u3_passes
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
pipeline: 512x512 field, 7 passes, 7 vkCmdPipelineBarrier2 calls
  GPU: 24174 cells selected, blurred sum 36600280, index xor 0x00027d33
  CPU: 24174 cells selected, blurred sum 36600280, index xor 0x00027d33
  GPU time 0.179 ms, median of 10 runs
cost: 256 dispatches of 16384 elements, GPU time, median of 10 runs
  sequence    barrier between dispatches             barriers    GPU ms us/dispatch
  chain       COMPUTE write → COMPUTE read/write         255     5.022       19.62
  chain       ALL_COMMANDS → ALL_COMMANDS                255     5.151       20.12
  independent none                                         0     1.397        5.46
  independent COMPUTE write → COMPUTE read/write         255     4.659       18.20
  independent ALL_COMMANDS → ALL_COMMANDS                255     5.470       21.37
  (a) precise, only where needed: 6.420 ms
  (b) full barrier after every dispatch: 10.621 ms
  (c) independent set, no barriers vs precise ones: 1.397 vs 4.659 ms
PASS
```

What a barrier costs

A barrier is where the GPU stops overlapping work. It must let every command in the source scope finish before anything in the destination scope starts, so the compute units drain, wait and fill again. With small dispatches, that bubble can cost more than the work. The second half of `u3_passes` measures it: 256 small dispatches, first as a chain in which each depends on the last, then as a set of independent dispatches writing separate ranges of a buffer, with precise barriers, with full `ALL_COMMANDS` barriers, and, for the independent set, with none.

The dispatches of the cost experiment, with a barrier of the chosen kind between them code/u3/passes/main.cpp

```
void recordSteps(VkCommandBuffer cmd, const u3::Kernel& step, VkDescriptorSet set,
                 uint32_t dispatches, uint32_t elements, bool chain, Barrier barrier) {
    const VkMemoryBarrier2 precise{
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
        .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .dstAccessMask =
            VK_ACCESS_2_SHADER_STORAGE_READ_BIT | VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    };
    const VkMemoryBarrier2 full{
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT,
        .srcAccessMask = VK_ACCESS_2_MEMORY_READ_BIT | VK_ACCESS_2_MEMORY_WRITE_BIT,
        .dstStageMask = VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT,
        .dstAccessMask = VK_ACCESS_2_MEMORY_READ_BIT | VK_ACCESS_2_MEMORY_WRITE_BIT,
    };
    const VkDependencyInfo dependency{
```

```
        .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
        .memoryBarrierCount = 1,
        .pMemoryBarriers = barrier == Barrier::Full ? &full : &precise,
    };
    vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, step.pipeline);
    const StepPush push{elements};
    vkCmdPushConstants(cmd, step.layout, VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(push), &push);
    for (uint32_t d = 0; d < dispatches; ++d) {
        if (d > 0 && barrier != Barrier::None) vkCmdPipelineBarrier2(cmd, &dependency);
        const uint32_t offset = chain ? 0 : d * elements * 4;
        vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, step.layout, 0, 1, &set,
                                1, &offset);
        vkCmdDispatch(cmd, vkf::groupCount(elements, 256), 1, 1);
    }
}
```

The independent dispatches select their range of the buffer with a *dynamic offset*: the descriptor's type is `VK_DESCRIPTOR_TYPE_STORAGE_BUFFER_DYNAMIC`, and each `vkCmdBindDescriptorSets` supplies an offset that is added to the binding's, so one descriptor set serves every range.

The cost experiment with validation off, median of 50 runs

```
$ VKF_VALIDATION=0 ./build/bin/u3_passes --repeats 50
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
pipeline: 512x512 field, 7 passes, 7 vkCmdPipelineBarrier2 calls
  GPU: 24174 cells selected, blurred sum 36600280, index xor 0x00027d33
  CPU: 24174 cells selected, blurred sum 36600280, index xor 0x00027d33
  GPU time 0.246 ms, median of 50 runs
cost: 256 dispatches of 16384 elements, GPU time, median of 50 runs
  sequence    barrier between dispatches             barriers    GPU ms us/dispatch
  chain       COMPUTE write → COMPUTE read/write         255     5.331       20.82
  chain       ALL_COMMANDS → ALL_COMMANDS                255     5.363       20.95
  independent none                                         0     0.819        3.20
  independent COMPUTE write → COMPUTE read/write         255     5.434       21.23
  independent ALL_COMMANDS → ALL_COMMANDS                255     5.771       22.54
  (a) precise, only where needed: 6.150 ms
  (b) full barrier after every dispatch: 11.134 ms
  (c) independent set, no barriers vs precise ones: 0.819 vs 5.434 ms
PASS
```

On the M2 Pro, a barrier between each pair of dispatches made every dispatch cost about 20 µs, whether the barrier was precise or covered all commands and all memory. Without barriers, the independent dispatches cost 3.2 µs each, because the GPU could run many at once. Precise barriers only where needed, a barrier between each dependent pair and none in the independent set, took 6.2 ms in all, and a full barrier after every dispatch took 11.1 ms.

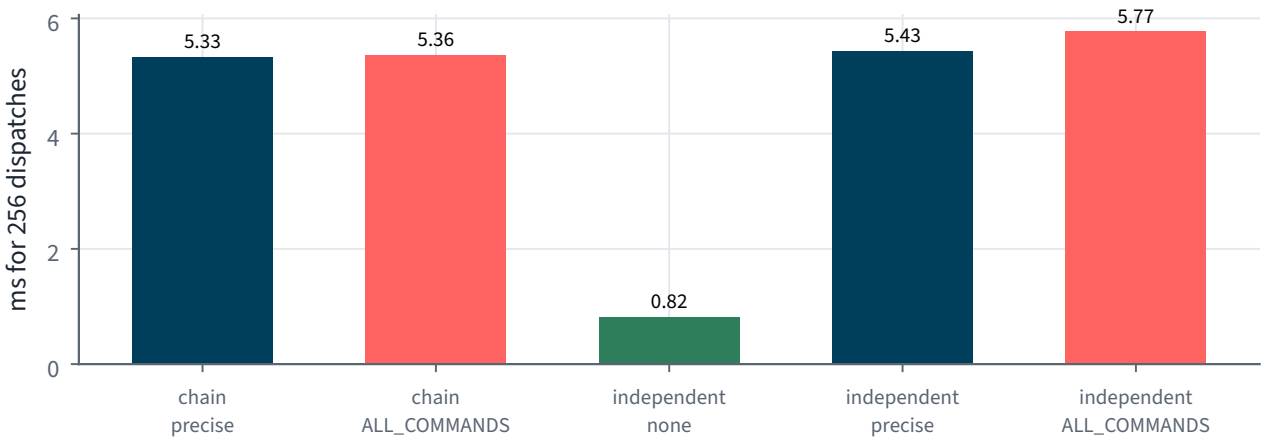


Figure 3.3.2. 256 dispatches of 16,384 elements on the M2 Pro. Any barrier makes each dispatch cost about 20 µs; without barriers, independent dispatches overlap and cost about 3 µs each.

The scope matters more on other GPUs and in other situations. A source scope of `ALL_COMMANDS` waits for every earlier command on the queue, including graphics work or copies that have nothing to do with the data, and `MEMORY_WRITE` with `MEMORY_READ` can make a driver flush and invalidate caches that a narrower barrier would leave alone. Lab 3.3.2 measures both on your GPU.

> KEY IDEA
> A barrier waits for all earlier work in its source stages, including work unrelated to the data it protects. Independent work recorded before a barrier is caught by it, and independent work recorded after it is held back. Minimise the number of barriers first, and their width second.

Placing and batching barriers

A few habits keep barriers few and narrow:

- **Batch.** Collect every barrier needed at one point into one `VkDependencyInfo`. One call lets the driver synchronise once.
- **Do not separate independent work.** Dispatches that touch different data need nothing between them, as the blur and edge passes show.
- **Prefer global barriers for buffers.** They cost the same as buffer barriers on most hardware, and cannot miss a buffer. Keep image barriers for layout transitions and ownership transfers.
- **Delay what you can.** A barrier placed just before the consumer gives the producer's work the longest time to overlap with whatever comes between, provided the barrier's source stages do not include that work. When they would, events (Chapter 3.4) or a second queue (Chapter 3.5) can separate the two.
- **Use a render graph.** A render graph derives and batches barriers from declarations of what each pass reads and writes. Chapter 3.8 builds one.

> COMING FROM OPENGL
> `glMemoryBarrier` is always global and always waits for every earlier command; `GL_ALL_BARRIER_BITS` is OpenGL's version of the full barrier above. Drivers also insert their own barriers between draws and dispatches when they detect a hazard, and a program cannot see or remove them. Vulkan's barriers cost something too, but every one is in your code, where you can count and measure it.

Reading

- The Vulkan Guide, "Synchronization Examples", the compute and transfer sections, and the graphics sections for Unit 4.
- The Vulkan specification, "Pipeline Barriers" and the valid-usage list of `vkCmdPipelineBarrier2`.
- NVIDIA, "Vulkan Dos and Don'ts", the section on barriers.

Labs

LAB 3.3.1 Justify every barrier — 1 hour

Goal. Read a real pipeline's barriers against the model.

Code. `code/u3/passes`

1. For each of the seven barriers in `Passes::record`, write the hazard it prevents, the producer and the consumer, and why each mask has the value it has.
2. Remove the second barrier of the batch before the selection, the one for the fill, and run with validation on.
3. Change the destination of the barrier before the score from `DRAW_INDIRECT` to `COMPUTE_SHADER`, keeping the access, and run again.

DONE WHEN
- [ ] Your notes justify every field of every barrier.
- [ ] You have recorded and explained both validation reports.

EVIDENCE FOR THE NOTEBOOK
- The seven justifications and the two reports.

LAB 3.3.2 Measure barriers on your GPU — 1.5 hours

Goal. Find out what barriers cost on your hardware.

Code. `code/u3/passes`

1. Run `VKF_VALIDATION=0 build/bin/u3_passes --repeats 50` three times and record the cost table.
2. Run again with `--elements 4096` and with `--elements 262144`, which set the elements per dispatch.
3. Plot the cost per dispatch against the elements per dispatch, with and without barriers.

DONE WHEN
- [ ] You have the plot, from three runs at each size.
- [ ] You can state at what dispatch size a barrier's cost stops mattering on your GPU, and whether its scope matters there.

EVIDENCE FOR THE NOTEBOOK
- The plot and your conclusion.

LAB 3.3.3 Independent work across a barrier — 1.5 hours

Goal. See a barrier delay independent work.

Code. `code/u3/passes`

1. Record a second, independent copy of the pipeline, on its own buffers and image, and time running the two one after the other.
2. Interleave the two copies pass by pass, keeping every barrier, and time again.
3. Explain the result with the definition of a barrier's source scope.

DONE WHEN
- [ ] Both versions produce correct results with validation silent.
- [ ] You can explain why interleaving did or did not help, and name the two features that could help.

EVIDENCE FOR THE NOTEBOOK
- The two timings and your explanation.

LAB 3.3.4 Batch and unbatch — 1 hour

Goal. Measure what batching saves.

Code. `code/u3/passes`

1. Split the batch before the selection into two `vkCmdPipelineBarrier2` calls, and time the pipeline with validation off.
2. Then merge the barriers before the filters and before the selection into one point, moving the edge pass after the blur pass's barrier, and explain why that is or is not still correct.

DONE WHEN

- [ ] You have timings for the batched and unbatched versions, from three runs each.
- [ ] Your explanation of the merged version uses the hazards involved.

EVIDENCE FOR THE NOTEBOOK

- The timings and the explanation.

Problem set

1. A compute pass writes a vertex buffer that the next draw reads. Write the `VkMemoryBarrier2`. Write it again for the same buffer used as an index buffer.
2. A render pass draws into an image that a compute pass then reads with a sampler. Write the image barrier, with its layouts.
3. Why does a barrier between two dependent dispatches also delay an unrelated dispatch recorded before it? What would change if the unrelated dispatch were recorded after the barrier?
4. On the M2 Pro each barrier between small dispatches cost about 20 µs. How many such barriers would fill a 60 Hz frame, and why is that the wrong way to budget them?
5. The batch before the selection holds two global barriers. Could it be one barrier? Write it, and say what it would cost or gain.

COMPLETION CHECKLIST

- [ ] I can find the dependency for any producer and consumer in the catalogue, including graphics ones.
- [ ] I can give a multi-pass pipeline exactly the barriers it needs, batched.
- [ ] I can explain what a barrier costs, and why its source scope catches unrelated work.
- [ ] I can measure barrier costs and judge when they matter.
- [ ] Lab 3.3.1–3.3.4 done-when criteria all hold, with evidence filed.

CHAPTER 3.4

Fences, semaphores and events

TIME	5 hours
YOU BUILD	A pipeline in which the host produces inputs and the GPU processes them, driven by one timeline semaphore with every submission made before its input exists, and compared with the same pipeline on fences; a chain of two submissions joined by a binary semaphore; and a split barrier with an event, measured against barriers in either of its two positions.
NEEDS	Chapters 3.1 to 3.3.

WHY IT MATTERS

Barriers order commands within a queue's stream of work. A program also needs to know when work has finished, to make one submission wait for another, and to let the host and the GPU hand data back and forth without either sitting idle. Vulkan has four primitives for these jobs, and choosing the wrong one costs either correctness or overlap. This chapter puts each to work and measures what it costs on the course's machine.

SKIP TEST

1. Which primitive lets the host wait for the GPU, which lets one submission wait for another, and which can do both?
2. The host writes a buffer after submitting the command buffer that reads it. What makes the GPU wait for the write, and what makes the write visible?
3. Why can a binary semaphore not be waited on before its signal has been submitted, when a timeline semaphore can?
4. What does a split barrier let the GPU do that a pipeline barrier does not?

If all four are easy, read the sections on timeline semaphores and events, and do Labs 3.4.1 and 3.4.3.

Core ideas

Four primitives

Table 3.4.1 summarises the four. A *fence* tells the host that a submission has finished. A *binary semaphore* makes one submission wait for another. A *timeline semaphore* does both. It holds a 64-bit counter that only increases, which submissions and the host can wait on and signal. An *event* splits a pipeline barrier in two within one queue.

Primitive	Signalled by	Waited on by	Typical use
Fence	A submission	The host	Knowing that a submission's resources can be reused
Binary semaphore	A submission, or presentation	One later submission	Ordering submissions across queues, and the swapchain (Chapter 4.2)
Timeline semaphore	Submissions or the host, with a value	Submissions or the host, for a value	Frames in flight, host and device pipelines, async compute
Event	A command	A later command in the same queue	Letting independent work run between a producer and its consumer

Table 3.4.1. Vulkan's synchronisation primitives beyond barriers

All four obey the dependency model of Chapter 3.2: a signal operation has a first synchronisation scope, the commands before it, and a wait has a second, the commands after it. The stage masks given with a semaphore name the stages that must finish before the signal, and the stages that must wait.

Fences

A fence is signalled when every command of the submission that names it has completed, and it makes all their memory writes available. The host waits for it with `vkWaitForFences`, and must reset it with `vkResetFences` before submitting with it again. The project's fence pipeline gives each of three slots a fence: it fills a slot's input, submits, and checks the slot's result three submissions later, when the slot comes round again.

A ring of three slots, each with its own fence — code/u3/timeline/main.cpp

```
for (uint32_t i = 0; i < items + ring.slots; ++i) {
    const uint32_t slot = i % ring.slots;
    if (i >= ring.slots) {
        vkf::waitFence(ctx, fences[slot]);
        VKF_CHECK(vkResetFences(ctx.device(), 1, fences[slot].ptr()));
        correct += ring.check(slot, i - ring.slots) ? 1 : 0;
    }
    if (i < items) {
        ring.fill(slot, i);
        vkf::submit(ctx.mainQueue(), std::span(&cmds[slot], 1), {}, {}, fences[slot]);
    }
}
```

The writes the host makes before `vkQueueSubmit2` need no barrier, because a submission makes every host write that came before it visible to the work it submits, once non-coherent memory has been flushed. The results need a barrier to the host stage in the command buffer. The fence orders the host's reads after the GPU's work, but only a barrier whose destination is the host makes the GPU's writes visible to it (Chapter 3.2).

Binary semaphores

A binary semaphore has two states. A submission signals it when the commands before the signal have completed, and a later submission waits for it, with the stages that must wait. Each signal is consumed by exactly one wait, and the wait must not be submitted before its signal is. Below, a semaphore joins two submissions on one queue so that the second reads what the first wrote. Across queues, the same pattern is the only way to order work (Chapter 3.5).

Two submissions joined by a binary semaphore — code/u3/timeline/main.cpp

```
const auto written = vkf::createSemaphore(ctx);
const auto fence = vkf::createFence(ctx);
const vkf::SemaphoreSubmit signal{written, 0, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT};
const vkf::SemaphoreSubmit wait{written, 0, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT};
vkf::submit(ctx.mainQueue(), std::span(&first, 1), {}, std::span(&signal, 1));
vkf::submit(ctx.mainQueue(), std::span(&second, 1), std::span(&wait, 1), {}, fence);
vkf::waitFence(ctx, fence);
```

A semaphore signal makes the writes of its first scope available, and the wait makes them visible to its second scope, so no barrier is needed between the two submissions.

Timeline semaphores

A timeline semaphore holds a counter. A wait names a value and completes when the counter reaches it. A signal names a value and sets the counter to it. Submissions signal and wait through `VkSemaphoreSubmitInfo`, and the host can do both too: `vkSignalSemaphore` sets the value from the CPU, and `vkWaitSemaphores` blocks until a value is reached, with a timeout. A timeline semaphore can replace both fences and binary semaphores, and one semaphore serves any number of submissions.

Unlike a binary semaphore, a timeline semaphore can be waited on before anything has been submitted to signal it, which is called *wait-before-signal*. The project uses it to submit each item's command buffer before the host has written that item's input. Item i waits for value $2i + 1$, which the host signals once the input is written, and signals $2i + 2$ when its result is ready:

Submitting an item, and signalling and waiting on the host — code/u3/timeline/main.cpp

```
// Item i waits for the host's value 2i + 1 and signals 2i + 2 when its result is ready.
const auto submitItem = [&](uint32_t item) {
    const vkf::SemaphoreSubmit ready{timeline, 2 * uint64_t(item) + 1,
                                     VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT};
    const vkf::SemaphoreSubmit done{timeline, 2 * uint64_t(item) + 2,
                                    VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT};
    vkf::submit(ctx.mainQueue(), std::span(&cmds[item % ring.slots], 1),
                std::span(&ready, 1), std::span(&done, 1));
};
const auto signal = [&](uint64_t value) {
    const VkSemaphoreSignalInfo info{
        .sType = VK_STRUCTURE_TYPE_SEMAPHORE_SIGNAL_INFO,
        .semaphore = timeline,
        .value = value,
    };
    VKF_CHECK(vkSignalSemaphore(ctx.device(), &info));
};
const auto waitFor = [&](uint64_t value) {
    const VkSemaphoreWaitInfo info{
        .sType = VK_STRUCTURE_TYPE_SEMAPHORE_WAIT_INFO,
        .semaphoreCount = 1,
        .pSemaphores = timeline.ptr(),
        .pValues = &value,
    };
    if (VKF_CHECK(vkWaitSemaphores(ctx.device(), &info, 5'000'000'000)) == VK_TIMEOUT) {
        throw std::runtime_error("timed out waiting for the timeline semaphore");
    }
};
```

Because the host writes after the submission, the submission does not cover those writes. The command buffer therefore begins with a barrier from the host's writes to the shader's reads, and the semaphore wait orders it after the host's signal:

An item's commands: from the host, the dispatch, back to the host — code/u3/timeline/main.cpp

```
void recordItem(VkCommandBuffer cmd, const Ring& ring, uint32_t slot, bool hostWritesLater) {
    // Host writes made after the submission are not covered by the submission itself.
    if (hostWritesLater) {
        const VkMemoryBarrier2 fromHost{
            .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
            .srcStageMask = VK_PIPELINE_STAGE_2_HOST_BIT,
            .srcAccessMask = VK_ACCESS_2_HOST_WRITE_BIT,
            .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
            .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
        };
        const VkDependencyInfo before{
            .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
            .memoryBarrierCount = 1,
            .pMemoryBarriers = &fromHost,
        };
        vkCmdPipelineBarrier2(cmd, &before);
    }
    u3::dispatch(cmd, ring.kernel, ring.sets[slot], CountPush{ring.count},
                 vkf::groupCount(ring.count, 256));
    toHost(cmd);
}
```

The loop keeps three items in flight. It submits each item as soon as its slot is free, fills the input, and signals when the input is ready:

Driving the pipeline from one timeline — code/u3/timeline/main.cpp

```
const uint32_t ahead = std::min(items, ring.slots);
for (uint32_t i = 0; i < ahead; ++i) submitItem(i);
for (uint32_t i = 0; i < ahead; ++i) ring.fill(i, i);
signal(1);
for (uint32_t i = 0; i < items; ++i) {
    waitFor(2 * uint64_t(i) + 2);
    if (i + 1 < items) signal(2 * uint64_t(i) + 3);
    correct += ring.check(i % ring.slots, i) ? 1 : 0;
    if (i + ring.slots < items) {
        submitItem(i + ring.slots);
        ring.fill(i % ring.slots, i + ring.slots);
    }
}
```

Two thousand items both ways, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u3_timeline --items 2000
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
timeline: 3 slots, 2000 items of 65536 values, each submitted before its input
  2000 of 2000 correct in 331.0 ms (6043 items per second)
fences: 3 slots, 2000 items, one fence per submission
  2000 of 2000 correct in 262.1 ms (7631 items per second)
binary semaphore: the second submission read the first's output: correct
PASS
```

The timeline version is correct but slower than the fences, at 6,043 items per second against 7,631, because of a rule about values. A timeline's value only increases, and the host may signal a value only if it is lower than every signal still pending on the GPU. While item i is pending with its signal of $2i + 2$, the host cannot signal $2i + 3$, so it must wait for item i before releasing item $i + 1$. With one semaphore carrying both directions, the host and the GPU take turns. Lab 3.4.1 gives each direction a timeline of its own.

Events

A pipeline barrier waits for its source scope and blocks its destination scope at one point in the command stream. An event splits that point in two: `vkCmdSetEvent2` marks where the source scope ends, `vkCmdWaitEvents2` marks where the destination scope begins, and commands recorded between them may run while the producer finishes. Both take the same `VkDependencyInfo`.

One dependency, used three ways code/u3/events/main.cpp

```
void record(VkCommandBuffer cmd, const Work& w, VkEvent event, Sync sync, uint32_t steps) {
    const VkMemoryBarrier2 produced{
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
        .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
    };
    const VkDependencyInfo dependency{
        .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
        .memoryBarrierCount = 1,
        .pMemoryBarriers = &produced,
    };
    u3::dispatch(cmd, w.spin, w.a, StepsPush{steps}, Groups);
    if (sync == Sync::Event) vkCmdSetEvent2(cmd, event, &dependency);
    if (sync == Sync::BarrierAtSet) vkCmdPipelineBarrier2(cmd, &dependency);
    u3::dispatch(cmd, w.spin, w.b, StepsPush{2 * steps}, Groups);
    if (sync == Sync::Event) vkCmdWaitEvents2(cmd, 1, &event, &dependency);
    if (sync == Sync::BarrierAtWait) vkCmdPipelineBarrier2(cmd, &dependency);
    u3::dispatch(cmd, w.spin, w.c, StepsPush{steps}, Groups);
}
```

The project runs three dispatches: A, B, which is independent of A and twice as long, and C, which reads A's output. A barrier where the set goes makes B wait for A. A barrier where the wait goes makes C wait for B as well as A. The event makes C wait for A alone. Events used only by commands are created with `VK_EVENT_CREATE_DEVICE_ONLY_BIT`, which tells the driver that the host never sets, resets or reads them:

A device-only event code/u3/events/main.cpp

```
vkf::Unique<VkEvent> createEvent(const vkf::Context& ctx) {
    // Device-only events are never set, reset or read by the host, which helps some drivers.
    const VkEventCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_EVENT_CREATE_INFO,
        .flags = VK_EVENT_CREATE_DEVICE_ONLY_BIT,
    };
    VkEvent event = VK_NULL_HANDLE;
    VKF_CHECK(vkCreateEvent(ctx.device(), &info, nullptr, &event));
    return {ctx.device(), event};
}
```

The three placements, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u3_events --repeats 20
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
A: 256 threads, 100000 steps; B: independent, 200000 steps; C: reads A's output
  barrier where the set goes (B waits for A)      10.607 ms
  barrier where the wait goes (C waits for B)     10.602 ms
  split barrier with an event (C waits for A)      8.095 ms
PASS: every variant matches the CPU (GPU time, median of 20 runs)
```

The event saves a quarter of the time. Its 8.1 ms is what A and B side by side, followed by C, would take: A and B overlap, but C still starts after B. Both barriers take the same 10.6 ms, although the barrier where the wait goes leaves A and B free to overlap. MoltenVK runs the dispatches of one Metal compute encoder one after another, and ends the encoder at each event command, which is what lets A overlap B here. On a desktop driver, the barrier where the wait goes may save as much as the event does here, and the event more. An event pays only where the hardware can overlap the work around it, so measure before keeping one.

> **PITFALL**
> On a portability implementation such as MoltenVK, events are one of the optional features of `VK_KHR_portability_subset`, and a feature is enabled only if the program chains

`VkPhysicalDevicePortabilitySubsetFeaturesKHR` into device creation with it set. The course's own `vkf` once enabled the extension without the structure, so every event it created was invalid, and the validation layer reported `VUID-vkCreateEvent-events-04468`. Enable the portability features you use, and run validation on a Mac before trusting code written elsewhere.

Waiting for idle

`vkQueueWaitIdle` and `vkDeviceWaitIdle` wait for everything submitted to a queue or a device. They are correct and simple, and right at shutdown, before destroying objects, and in tests. In a loop they serialise the CPU and the GPU: the host waits for all work instead of the work it needs, and nothing is queued while it waits. Prefer a fence or a timeline value that names exactly the work you depend on.

KEY IDEA

Use a fence or a timeline value to know when work is done, a semaphore to order submissions, and an event to let independent work run between a producer and its consumer. Host writes made before a submission are covered by it. Host writes made after it need a semaphore to order them and a barrier to make them visible.

COMING FROM OPENGL

`glFenceSync` and `glClientWaitSync` are OpenGL's fences, and `glWaitSync` makes the server wait for one, which is as close as OpenGL comes to a semaphore. There is no counterpart to timeline semaphores or events: the driver decides how much work overlaps, and a program cannot submit work before its inputs exist.

Reading

- The Vulkan specification, "Synchronization and Cache Control": the sections on fences, semaphores and events.
- The Khronos Group, "Vulkan Timeline Semaphores", 2019: the extension's introduction, with examples.
- The Khronos wiki page "Synchronization Examples" in the Vulkan-Docs repository, in particular the sections on semaphores and events.

Labs

LAB 3.4.1 Two timelines — 1.5 hours

Goal. Let the host and the GPU run independently.

Code. `code/u3/timeline`

1. Replace the single timeline with two: one the host signals when an input is ready, and one the GPU signals when a result is ready.
2. Measure items per second with two, three and four slots, with validation off.

DONE WHEN

- [ ] Every item is correct, and validation is silent.
- [ ] Your version beats the single timeline, and you can explain how it compares with the fences.

EVIDENCE FOR THE NOTEBOOK

- The changes and the table.

LAB 3.4.2 Wait before signal — 1 hour

Goal. See a wait that no submission can satisfy.

Code. `code/u3/timeline`

1. Submit one item that waits for a timeline value, without signalling it, and call `vkWaitSemaphores` for the item's result with a timeout of one second.
2. Then signal the value from the host and wait again.
3. Try the same with a binary semaphore that nothing has signalled, with validation on.

DONE WHEN

- [ ] The first wait times out and the second succeeds.
- [ ] You have the validation message from the binary semaphore, and can explain why the rule differs between the two kinds.

EVIDENCE FOR THE NOTEBOOK

- The output and the message.

LAB 3.4.3 Where events pay — 1 hour

Goal. Predict when a split barrier saves time.

Code. `code/u3/events`

1. Change the program so that B's work is half of, equal to and four times A's.
2. Time A, B and C alone, predict each variant's time from those, and compare with the measurements.

DONE WHEN

- [ ] Your predictions are within 10% of the measurements.
- [ ] You can explain why the two barrier placements take the same time on your device, or why they do not.

EVIDENCE FOR THE NOTEBOOK

- The table of predictions and measurements.

LAB 3.4.4 What idling costs — 1 hour

Goal. Measure an idle wait against a fence.

Code. `code/u3/timeline`

1. In the fence pipeline, replace the wait for each slot's fence with `vkQueueWaitIdle` after every submission.
2. Measure items per second against the original.

DONE WHEN

- [ ] Both versions are correct, with validation silent.
- [ ] You can explain the difference from what each wait lets the host do while the GPU works.

EVIDENCE FOR THE NOTEBOOK

- The two measurements and the explanation.

Problem set

1. A fence is submitted a second time without being reset. What does the validation layer report, and what would a driver do without it?

2. Two submissions wait on the same signal of one binary semaphore. What is wrong, and how would you express the same dependency correctly?
3. In the timeline loop, why can the host not signal $2i + 3$ as soon as it has written item $i + 1$'s input?
4. `vkCmdWaitEvents2` must be given the same `VkDependencyInfo` as `vkCmdSetEvent2`. Why does the wait need the source scope at all?
5. When is `vkDeviceWaitIdle` the right call, and what does it cost inside a frame loop?

COMPLETION CHECKLIST

- ☐ I can choose between fences, binary and timeline semaphores, and events for each kind of dependency.
- ☐ I can drive a host and device pipeline from timeline semaphores, including waits submitted before their signals.
- ☐ I can tell when a submission covers host writes, and when they need a semaphore and a barrier.
- ☐ I can split a barrier with an event, and measure whether it pays on my device.
- ☐ Lab 3.4.1–3.4.4 done-when criteria all hold, with evidence filed.

CHAPTER 3.5

Queues and async compute

TIME	6 hours
YOU BUILD	An upload on a transfer queue whose buffer is handed to the main queue with a queue family ownership transfer, and the same upload with a buffer shared concurrently by both families; and a measurement of two independent workloads on one queue and on two, which shows when a second queue doubles the throughput and when it barely helps.
NEEDS	Chapters 3.1 to 3.4.

WHY IT MATTERS

Most GPUs expose several queues, often in families with different abilities: one for everything, one for compute and copies, one for copies alone. Work on different queues can run at the same time, which lets a frame overlap its simulation with its drawing, or stream data in while it renders. The cost is that Vulkan's ordering guarantees end at the boundary of a queue. Between queues there is only what semaphores provide, and a resource used by two queue families must be handed from one to the other explicitly.

SKIP TEST

1. Two queues of different families use one buffer in turn. What does an `EXCLUSIVE` buffer need between the uses that a `CONCURRENT` one does not?
2. A release barrier is recorded on the transfer queue and an acquire on the main queue. What orders the acquire after the release?
3. When does a second queue make independent work finish sooner, and when does it not?
4. Why can a timestamp written on one queue not be compared with one written on another?

If all four are easy, read the section on measuring overlap and do Labs 3.5.2 and 3.5.3.

Core ideas

Queue families

Chapter 1.1 listed the device's queue families: each has flags saying which commands its queues accept, and a number of queues. Graphics and compute commands need a family with those flags; any family that supports either also supports transfers. On a desktop GPU, a typical arrangement is one family for everything, a family of compute-only queues for *async compute*, and a transfer-only family whose queues drive the copy engines. The M2 Pro has four families of one queue each, all able to do everything.

The project asks `vkf` for a second compute queue and a transfer queue, in families other than the main queue's where the device has them:

The project on the M2 Pro: three families, two uploads and the overlap, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u3_queues --repeats 20
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
queues: main family 0, transfer family 2, compute family 1
upload, exclusive buffer: released by family 2, acquired by family 0, correct
upload, concurrent buffer: no ownership transfer, correct
two independent workloads, ms from submit to finish, median of 20
  workload               one queue     + barrier  two queues   GPU ms: main, compute
  8 groups x 400000         21.410        21.411      10.983          10.597, 10.701
  2048 groups x 4000         2.979         3.019       2.763           2.050,  1.796
PASS
```

Commands submitted to one queue begin in submission order, and the barriers of Chapter 3.2 order them. Commands on different queues have no order at all unless a semaphore gives them one, and a barrier on one queue says nothing about another.

Sharing a resource between families

A buffer or image is created with a *sharing mode*. An `EXCLUSIVE` resource is owned by one queue family at a time, and the program must transfer ownership to use its contents in another family. A `CONCURRENT` resource lists the families that will use it and can be used by any of them without a transfer. Concurrent sharing is simpler and may be slower: a driver can lay out or compress an exclusive image for one kind of hardware, which concurrent sharing prevents.

A buffer shared by two families — code/u3/queues/main.cpp

```
ConcurrentBuffer createConcurrent(const vkf::Context& ctx, VkDeviceSize size,
                                  VkBufferUsageFlags usage, uint32_t familyA,
                                  uint32_t familyB) {
    const uint32_t families[] = {familyA, familyB};
    const VkBufferCreateInfo info{
        .sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO,
        .size = size,
        .usage = usage,
        .sharingMode = VK_SHARING_MODE_CONCURRENT,
        .queueFamilyIndexCount = 2,
        .pQueueFamilyIndices = families,
    };
```

A *queue family ownership transfer* is a pair of barriers with the same family indices, the same resource and, for an image, the same layout transition. The *release* is recorded on the source queue: its source scope is the work that wrote the resource, and its destination scope is empty, because nothing on that queue uses the resource afterwards.

Copying on the transfer queue, then releasing the buffer to the main queue's family — code/u3/queues/main.cpp

```
void recordCopy(VkCommandBuffer cmd, VkBuffer staging, VkBuffer target, VkDeviceSize bytes,
                uint32_t from, uint32_t to) {
    const VkBufferCopy region{.size = bytes};
    vkCmdCopyBuffer(cmd, staging, target, 1, &region);
    if (from == to) return;
    const VkBufferMemoryBarrier2 release{
        .sType = VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
        .srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
        .dstStageMask = VK_PIPELINE_STAGE_2_NONE,
        .dstAccessMask = VK_ACCESS_2_NONE,
        .srcQueueFamilyIndex = from,
        .dstQueueFamilyIndex = to,
        .buffer = target,
        .offset = 0,
        .size = VK_WHOLE_SIZE,
    };
    const VkDependencyInfo dependency{
        .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
        .bufferMemoryBarrierCount = 1,
        .pBufferMemoryBarriers = &release,
    };
    vkCmdPipelineBarrier2(cmd, &dependency);
}
```

A semaphore then orders the two queues, and the *acquire* is recorded on the destination queue. Its destination scope is the work that will use the resource. Its source scope matches the semaphore's wait stage, so that the acquire is chained after the wait and therefore after the release.

Acquiring the buffer on the main queue before reading it — code/u3/queues/main.cpp

```
void recordUse(VkCommandBuffer cmd, const u3::Kernel& use, VkDescriptorSet set,
               VkBuffer target, uint32_t count, uint32_t from, uint32_t to) {
    if (from != to) {
        // The source stage matches the semaphore wait, so the acquire happens after it.
        const VkBufferMemoryBarrier2 acquire{
            .sType = VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2,
            .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
            .srcAccessMask = VK_ACCESS_2_NONE,
            .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
            .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
            .srcQueueFamilyIndex = from,
            .dstQueueFamilyIndex = to,
            .buffer = target,
            .offset = 0,
            .size = VK_WHOLE_SIZE,
        };
        const VkDependencyInfo dependency{
            .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
            .bufferMemoryBarrierCount = 1,
            .pBufferMemoryBarriers = &acquire,
        };
        vkCmdPipelineBarrier2(cmd, &dependency);
    }
    u3::dispatch(cmd, use, set, CountPush{count}, vkf::groupCount(count, 256));
    vkf::memoryBarrier(cmd, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                       VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT, VK_PIPELINE_STAGE_2_HOST_BIT,
                       VK_ACCESS_2_HOST_READ_BIT);
}
```

A semaphore between the transfer queue and the main queue code/u3/queues/main.cpp

```
const auto uploaded = vkf::createSemaphore(ctx);
const auto fence = vkf::createFence(ctx);
const vkf::SemaphoreSubmit signal{uploaded, 0, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT};
vkf::submit(ctx.transferQueue(), std::span(&copy, 1), {}, std::span(&signal, 1));
const vkf::SemaphoreSubmit wait{uploaded, 0, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT};
vkf::submit(ctx.mainQueue(), std::span(&read, 1), std::span(&wait, 1), {}, fence);
vkf::waitFence(ctx, fence);
```

The release and the acquire together form a dependency: the release makes the copy's writes available, and the acquire makes them visible to the compute shader. The semaphore's signal stage, `ALL_COMMANDS`, covers the release, which executes as part of the barrier on the transfer queue.

PITFALL

An ownership transfer preserves contents. A resource whose old contents will not be read, because the next family overwrites it, needs no transfer. Without one, its contents are undefined in the new family, and an overwrite does not need them. A resource whose contents matter and that skips the transfer can work on one GPU and show corrupted data on another. The validation layer reports an acquire with no matching release when the work is submitted, but cannot see a use that should have had both.

Async compute

The project's second experiment runs two independent workloads, each a dispatch whose invocations repeat a dependent chain of arithmetic. Each runs once on one queue, once on one queue with an execution barrier between them, and once on two queues, one dispatch each.

One dispatch on the main queue and one on the compute queue code/u3/queues/main.cpp

```
VkCommandBuffer onMain = vkf::allocateCommandBuffer(ctx, mainPool);
vkf::beginCommands(onMain);
mainTimer.reset(onMain);
mainTimer.stamp(onMain);
u3::dispatch(onMain, work, firstSet, StepsPush{w.steps}, w.groups);
mainTimer.stamp(onMain);
toHost(onMain);
vkf::endCommands(onMain);
VkCommandBuffer onCompute = vkf::allocateCommandBuffer(ctx, computePool);
vkf::beginCommands(onCompute);
if (computeTimed) {
    computeTimer.reset(onCompute);
    computeTimer.stamp(onCompute);
}
u3::dispatch(onCompute, work, secondSet, StepsPush{w.steps}, w.groups);
if (computeTimed) computeTimer.stamp(onCompute);
toHost(onCompute);
vkf::endCommands(onCompute);
vkf::CpuTimer wall;
vkf::submit(ctx.mainQueue(), std::span(&onMain, 1), {}, {}, mainFence);
vkf::submit(ctx.computeQueue(), std::span(&onCompute, 1), {}, {}, computeFence);
finish(mainFence);
finish(computeFence);
```

With 8 workgroups, each workload occupies a small part of the GPU and takes 10.6 ms. On one queue they take 21.4 ms, whether or not a barrier separates them, because MoltenVK runs the dispatches of one queue's Metal encoder one after another, whatever the barriers allow. On two queues they take 11.0 ms, almost exactly one workload's time. With 2,048 workgroups, each workload fills the GPU by itself, and two queues save only about 8%, because little hardware is left idle for the second queue to use.

A second queue pays when the work on the first leaves hardware idle: a shadow pass that rasterises without shading, a frame's tail of small dispatches, a simulation that cannot fill the GPU alone. It pays little when both workloads are limited by the same resource, such as memory bandwidth, and it can slow the first queue's work by competing for caches. On a desktop driver, the two dispatches on one queue may overlap

without a barrier, so the gap between one queue and two may be smaller there. Measure on the hardware you ship for.

`VkDeviceQueueCreateInfo` takes a priority between 0 and 1 for each queue, which drivers may use when queues compete. Two queues of one family may or may not map to separate hardware queues. Separate families are a better sign that they do.

Measuring across queues

Timestamps can only be compared when they were written on the same queue, so the project times each queue's work on that queue, and the overlap with the CPU's clock: from the first submission to the last fence. `VK_KHR_calibrated_timestamps` relates each queue's timestamps to a CPU clock where a finer comparison is needed.

> **KEY IDEA**
> Commands on different queues are ordered only by semaphores. An `EXCLUSIVE` resource whose contents must survive a change of family needs a release on one queue and a matching acquire on the other, chained by the semaphore's wait stage. A second queue makes independent work finish sooner only when the first leaves hardware idle.

> **COMING FROM OPENGL**
> OpenGL has one implicit queue per context. Several contexts can share objects and be used from several threads, but the driver decides whether their work runs concurrently, and none of it is exposed: there are no queue families, no ownership and no way to put compute beside graphics deliberately.

Reading

- The Vulkan specification, "Queue Family Ownership Transfer" and "Resource Sharing Mode".
- The Khronos wiki page "Synchronization Examples": the sections on transfers between queue families and on async compute.
- AMD GPUOpen, "Concurrent execution: asynchronous queues", on what async compute overlaps on AMD hardware.

Labs

LAB 3.5.1 Map your queues — 1 hour

Goal. List the queue families your GPU offers.

Code. `code/u3/queues`

1. List your device's queue families with `u1_devices` or `vulkaninfo`, with their flags, queue counts and `timestampValidBits`.
2. Run `u3_queues` and `u3_queues --shared-queue` and note which families `vkf` chose.

DONE WHEN

- [ ] You have a table of your families.
- [ ] You can say which of them allow async compute and transfers in parallel, and what `vkf` picks when a device has only one family.

EVIDENCE FOR THE NOTEBOOK

- The table and the two outputs.

LAB 3.5.2 Hand over an image — 1.5 hours

Goal. Transfer ownership with a layout transition.

Code. `code/u3/queues`

1. Upload an image on the transfer queue: transition it from `UNDEFINED` for the copy, copy from a buffer, and release it to the main family with a transition to `GENERAL`.
2. Acquire it on the main queue with the same transition, and read it with `imageLoad` in a compute shader.

DONE WHEN

- [ ] The values read match the uploaded ones, and validation is silent.
- [ ] You can explain on which queue the transition happens, and why both barriers must name it.

EVIDENCE FOR THE NOTEBOOK

- The code and the output.

LAB 3.5.3 Find the overlap — 1.5 hours

Goal. Measure where a second queue stops helping.

Code. `code/u3/queues`

1. Run the overlap experiment with 8, 32, 128, 512 and 2,048 workgroups, keeping each workload's total work constant by adjusting the steps.
2. Plot the time on two queues against the time on one.

DONE WHEN

- [ ] You have the plot.
- [ ] You can explain where the gain disappears in terms of how many workgroups your GPU can run at once.

EVIDENCE FOR THE NOTEBOOK

- The plot and the explanation.

LAB 3.5.4 Break the transfer — 1 hour

Goal. Learn what validation sees.

Code. `code/u3/queues`

1. Remove the release, keeping the acquire, and run with validation on.
2. Restore it, remove the acquire instead, and run again.

DONE WHEN

- [ ] You have both outputs.
- [ ] You can explain why one mistake is reported and the other is not, and what the second could do on a GPU that compresses buffers or caches them per family.

EVIDENCE FOR THE NOTEBOOK

- The two outputs and the explanation.

Problem set

1. Why must the acquire barrier's source stage match the semaphore's wait stage? What happens if it is `NONE`?

2. A buffer written on the transfer queue is overwritten entirely by a compute shader on the main queue, which never reads the old contents. Does it need an ownership transfer? A semaphore?
3. How would you measure, on the GPU's clock, how long the two queues' work overlapped?
4. Two workloads limited by memory bandwidth run on two queues. Predict the result, and explain it with Chapter 2.5's roofline.
5. A device has one family with 16 queues that support everything. Is creating two of them async compute? How would you find out?

COMPLETION CHECKLIST

- [ ] I can choose queues from the device's families and order their work with semaphores.
- [ ] I can transfer ownership of a buffer or image between families, and know when no transfer is needed.
- [ ] I can choose between exclusive and concurrent sharing, and say what each costs.
- [ ] I can measure whether async compute pays for a workload on my GPU.
- [ ] Lab 3.5.1–3.5.4 done-when criteria all hold, with evidence filed.

CHAPTER 3.6

Frames in flight

TIME	5 hours
YOU BUILD	A loop that renders frames without a window, in which the CPU prepares each frame's inputs and the GPU computes an image from them, run with one, two and three frames in flight. Per-frame resources live in rings, one timeline semaphore paces the loop, a palette replaced every sixteen frames is destroyed only when no frame can still read it, and every frame's image is checked against a checksum the CPU computed.
NEEDS	Chapters 3.1 to 3.4.

WHY IT MATTERS

A real-time program has two processors working on every frame. If the CPU waits for each frame to finish before preparing the next, the GPU idles while the CPU works and the CPU idles while the GPU works, and the frame takes the sum of both. Keeping more than one frame *in flight* lets each processor work on a different frame at once, so that a frame takes only as long as the slower of the two. The cost is that every resource the CPU writes for a frame must exist once per frame in flight, and nothing may be destroyed while a frame in flight can still use it.

SKIP TEST

1. With two frames in flight, which per-frame resources need two copies, and which can be shared?
2. Before reusing the resources of frame $n - 2$, what must the CPU wait for, and how does one timeline semaphore express it?
3. A buffer that frames still in flight may read must be replaced. When can the old one be destroyed?
4. What does a third frame in flight buy over a second, and what does it cost?

If all four are easy, read the section on deferred destruction and do Labs 3.6.2 and 3.6.3.

Core ideas

Pipelining the CPU and the GPU

The project's frames are a stand-in for a renderer's: for each frame, the CPU spends 1.8 ms generating 16,384 seeds and the checksum the frame's image must have, and the GPU spends 1.1 ms turning the seeds into a 256 × 256 image, which is copied back to the host. With one frame in flight the CPU prepares a frame, submits it, and waits for it before preparing the next, so the two processors take turns (Figure 3.6.1).

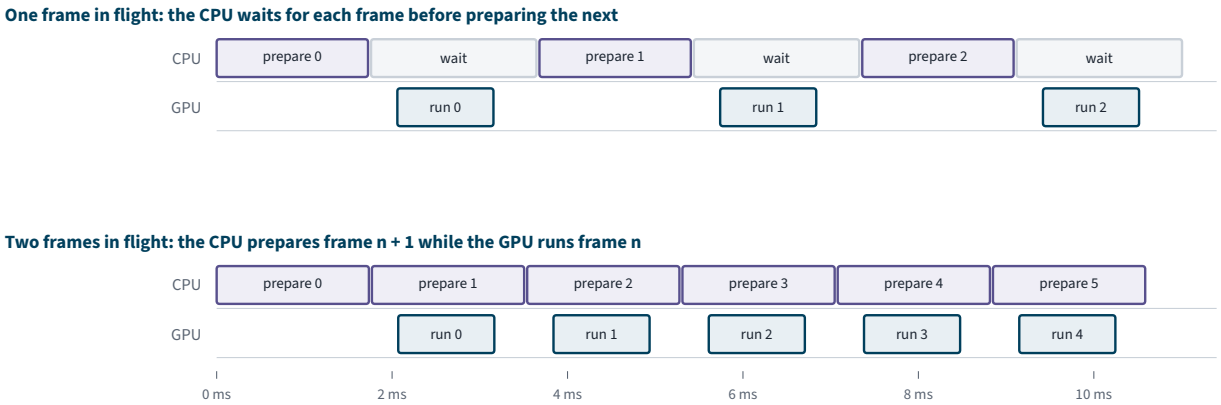


Figure 3.6.1. The measured frame times as a timeline. With one frame in flight a frame costs the CPU's time plus the GPU's, and more; with two it costs the larger of the two.

Three hundred frames with one, two and three in flight, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u3_frames --frames 300
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
300 frames: CPU prepares 16384 seeds, GPU runs 8000 steps per pixel of 256x256
  in flight     ms/frame    CPU work   CPU waits         GPU     checksums
  1               3.643       1.754       1.768       1.133       300/300
            19 palettes, each destroyed at most 1 frame(s) after it was replaced
  2               1.828       1.757       0.000       1.274       300/300
            19 palettes, each destroyed at most 1 frame(s) after it was replaced
  3               1.770       1.749       0.000       1.343       300/300
            19 palettes, each destroyed at most 1 frame(s) after it was replaced
PASS
```

With one frame in flight a frame takes 3.6 ms: 1.8 ms of CPU work, and 1.8 ms of waiting, which is the GPU's 1.1 ms plus the time to submit the work, have it start, and learn that it finished. With two in flight it takes 1.8 ms, the CPU's work alone, and the CPU never waits. The CPU is now the limit, and the GPU idles a third of the time. A third frame changes nothing, because nothing is left to overlap.

Rings of per-frame resources

Everything the CPU writes for a frame must not be written again while the GPU may still be reading it. With F frames in flight, each such resource becomes a *ring* of F copies, and frame f uses copy f mod F. Here that means the seeds, the readback buffer, the command buffer, the GPU timer and the descriptor set; in a renderer, the uniform buffers and the per-frame descriptor sets as well. Resources that only the GPU touches, one frame after another, can be shared. The project's image is one of these. Each frame overwrites it, and a barrier orders the overwrite after the previous frame's copy, which works because all the frames are in submission order on one queue.

Each frame discards the previous frame's image after its copy has read it — code/u3/frames/main.cpp

```
slot.timer.reset(cmd);
slot.timer.stamp(cmd);
// The previous frame's copy may still be reading the image; this frame replaces it.
VkImageMemoryBarrier2 image{
    .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
    .srcAccessMask = VK_ACCESS_2_NONE,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .oldLayout = VK_IMAGE_LAYOUT_UNDEFINED,
    .newLayout = VK_IMAGE_LAYOUT_GENERAL,
    .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .image = image_,
    .subresourceRange = vkf::colorRange(),
};
const VkDependencyInfo dependency{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .imageMemoryBarrierCount = 1,
    .pImageMemoryBarriers = &image,
};
vkCmdPipelineBarrier2(cmd, &dependency);
u3::dispatch(cmd, kernel_, slot.set, Push{Steps, SeedCount}, Side / 16, Side / 16);
```

Copying the frame's image into the frame's slot of the readback ring — code/u3/frames/main.cpp

```
image.srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT;
image.srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT;
image.dstStageMask = VK_PIPELINE_STAGE_2_COPY_BIT;
image.dstAccessMask = VK_ACCESS_2_TRANSFER_READ_BIT;
image.oldLayout = VK_IMAGE_LAYOUT_GENERAL;
image.newLayout = VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL;
vkCmdPipelineBarrier2(cmd, &dependency);
const VkBufferImageCopy region{
    .bufferOffset = VkDeviceSize(s) * Side * Side * 4,
    .imageSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
    .imageExtent = {Side, Side, 1},
};
vkCmdCopyImageToBuffer(cmd, image_, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, readback_, 1,
                       &region);
vkf::memoryBarrier(cmd, VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
                   VK_PIPELINE_STAGE_2_HOST_BIT, VK_ACCESS_2_HOST_READ_BIT);
slot.timer.stamp(cmd);
```

Pacing with a timeline semaphore

One timeline semaphore paces the loop. Frame f's submission signals the value $f + 1$ when it completes. Before frame f reuses slot $f \bmod F$, the CPU waits for the value $f - F + 1$, which says that frame $f - F$, the last user of the slot, has finished. The CPU can then check that frame's result and overwrite its resources.

Waiting for the frame that last used this slot, then checking its result — code/u3/frames/main.cpp

```
const uint32_t s = f % inFlight_;
Slot& slot = slots_[s];
vkf::CpuTimer waiting;
if (f ⩾ inFlight_) waitFor(f - inFlight_ + 1);
waits.push_back(waiting.elapsedMs());
vkf::CpuTimer working;
if (slot.frame ⩾ 0) finish(slot, s);
```

Re-recording the slot's command buffer and submitting the frame — code/u3/frames/main.cpp

```
VKF_CHECK(vkResetCommandBuffer(slot.cmd, 0));
vkf::beginCommands(slot.cmd);
record(slot.cmd, slot, s);
vkf::endCommands(slot.cmd);
const vkf::SemaphoreSubmit done{timeline_, f + 1,
                                VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT};
vkf::submit(ctx_.mainQueue(), std::span(&slot.cmd, 1), {}, std::span(&done, 1));
slot.frame = f;
```

The command buffer is re-recorded every frame, because its descriptor set changes with the palette. A program whose commands never change could record one per slot once. The descriptor set itself is updated in place, which is allowed because the frame that last used it has finished.

Deferred destruction

Every sixteen frames the program replaces the palette that the shader reads. The old palette cannot be destroyed at once, because frames already submitted may still read it. It is *retired* instead: kept, with the timeline value after which no frame can use it, and destroyed when the semaphore passes that value.

A resource waiting for the frames that may use it — code/u3/frames/main.cpp

```
// A resource that frames still in flight may use, kept until the timeline passes `after`.
struct Retired {
    uint64_t after;
    vkf::Buffer buffer;
};
```

Destroying what the GPU has finished with, and retiring the old palette — code/u3/frames/main.cpp

```
uint64_t completed = 0;
VKF_CHECK(vkGetSemaphoreCounterValue(ctx_.device(), timeline_, &completed));
while (!retired_.empty() && retired_.front().after <= completed) {
    result.longestWait =
        std::max(result.longestWait, f - uint32_t(retired_.front().after));
    retired_.pop_front();
}
if (f > 0 && f % PaletteFrames == 0) {
    // Frames up to f - 1 may still read the old palette; frame f - 1 signals f.
    retired_.push_back({f, std::move(palette_)});
    newPalette(f / PaletteFrames);
}
```

The same pattern serves every resource a renderer replaces while it runs: textures streamed out, buffers resized, pipelines rebuilt. The program reports that each palette lived at most one frame beyond its replacement, which is as soon as the timeline allows.

How many frames in flight

Two frames in flight are enough when the CPU and the GPU each take less than a frame's budget. A third hides variation: when one frame's CPU work is unusually long, the GPU still has a frame queued. Each frame in flight costs memory for its ring of resources and, for an interactive program, latency: the image on screen reflects input sampled one frame earlier for every frame queued ahead of it. Chapter 4.2 adds presentation, whose own images and semaphores interact with this loop.

> **KEY IDEA**
> Give everything the CPU writes for a frame a ring of as many copies as there are frames in flight, pace the loop with one timeline semaphore whose value counts finished frames, and retire replaced resources until the semaphore passes the last frame that could use them.

COMING FROM OPENGL
An OpenGL driver keeps frames in flight on the program's behalf: it buffers commands, renames buffers that the program overwrites with `glBufferData`, and blocks in `SwapBuffers` when it is too far ahead. Persistently mapped buffers, from OpenGL 4.4, make the program responsible again, with `glFenceSync` playing the part of the timeline semaphore.

Reading

- The Vulkan Guide, "Synchronization", on frames in flight with a swapchain.
- Hans-Kristian Arntzen, "Yet another blog explaining Vulkan synchronization", 2019, on the frame loop's dependencies.
- The Khronos wiki page "Synchronization Examples": the section on swapchain image acquire and present.

Labs

LAB 3.6.1 Who limits the frame — 1 hour

Goal. Make each processor the bottleneck in turn.
Code. `code/u3/frames`

1. Run with `--cpu-rounds` set to 8, 48 and 96, with validation off.
2. For each, record the frame time with one, two and three frames in flight.

DONE WHEN
- [ ] You have a table of nine frame times.
- [ ] You can predict each from the CPU and GPU columns, and say when a third frame in flight helps.

EVIDENCE FOR THE NOTEBOOK
- The table and the predictions.

LAB 3.6.2 Break the ring — 1 hour

Goal. See what happens when a frame overwrites another's inputs.
Code. `code/u3/frames`

1. Make every frame write its seeds to the first slot of the seed ring, and bind that slot in its descriptor set.
2. Run with two frames in flight and `--cpu-rounds 1`, so that the CPU writes the next frame's seeds while the GPU still runs the current one, with validation on and then off.

DONE WHEN
- [ ] You have the checksum counts from both runs.
- [ ] You can explain why the validation layer says nothing, and why the damage may vary from run to run.

EVIDENCE FOR THE NOTEBOOK
- The outputs and the explanation.

LAB 3.6.3 Destroy too soon — 1 hour

Goal. Find an off-by-one in deferred destruction.

Code. `code/u3/frames`

1. Retire the old palette with the value $f - 1$ instead of f, and run with three frames in flight and validation on.

DONE WHEN

- [ ] You have the validation message.
- [ ] You can explain which frame could still read the palette, and why the message names a command buffer.

EVIDENCE FOR THE NOTEBOOK

- The message and the explanation.

LAB 3.6.4 Measure the latency — 1.5 hours

Goal. Measure the latency that frames in flight add.

Code. `code/u3/frames`

1. Record, for each frame, the CPU time at which its seeds were prepared and the time at which its result was checked.
2. Report the median latency with one, two and three frames in flight.

DONE WHEN

- [ ] You have the three latencies.
- [ ] You can explain why the latency grows with frames in flight while the frame time does not.

EVIDENCE FOR THE NOTEBOOK

- The changes and the numbers.

Problem set

1. With two frames in flight the CPU never waits, and the frame takes 1.8 ms. Which processor limits the frame rate, and what would make a third frame useful?
2. Why can the image be shared by every frame while the seeds need a ring? What would change if the GPU wrote the seeds?
3. The palette is retired with the value f. Why not $f + 1$, and why not $f - 1$?
4. Show that waiting for the value $f - F + 1$, with $F = 1$, is waiting for the previous frame to finish.
5. A game samples input at the start of each frame and runs three frames in flight at 60 frames per second. Estimate the delay between input and its effect on screen, before presentation adds its own.

COMPLETION CHECKLIST

- [ ] I can pipeline the CPU and the GPU with frames in flight and measure the result.
- [ ] I can decide which resources need a ring and which can be shared.
- [ ] I can pace a frame loop with one timeline semaphore.
- [ ] I can defer the destruction of replaced resources until no frame can use them.
- [ ] Lab 3.6.1–3.6.4 done-when criteria all hold, with evidence filed.

CHAPTER 3.7

Finding synchronisation bugs

TIME	6 hours
YOU BUILD	A catalogue of six classic synchronisation bugs, each a small program that fails the way real programs do, with what the validation layer said about it and whether its results went wrong; their fixes; and a field guide to what synchronisation validation cannot see, measured on the course's machine, with a method for the bugs it misses.
NEEDS	Chapters 3.1 to 3.6.

WHY IT MATTERS

A program with a synchronisation bug usually gives the right answer on the machine it was written on, because that GPU happened to finish one command before starting the next, and fails on another GPU, on a newer driver, or one frame in a thousand. Testing results is therefore not enough. This chapter collects the bugs that recur in real programs, shows how each is found, and maps the places where the tools are blind, so that you know when a silent validation layer proves something and when it does not.

SKIP TEST

1. A barrier names the right stages and no access masks. What is wrong, and will the results show it?
2. Which pipeline stage does `vkCmdFillBuffer` run in?
3. A semaphore wait names the compute stage, and the first command after it is a copy. What can go wrong?
4. Name three kinds of synchronisation error that the validation layer cannot report.

If all four are easy, read the section on blind spots and do Labs 3.7.2 and 3.7.4.

Core ideas

Bugs that pass their tests

`u3_bugs` contains six programs, each with a classic mistake that `--bug` switches on. Table 3.7.1 summarises what happened to each on the M2 Pro.

Bug	The mistake	Found by	Results
missing-visibility	A barrier with the right stages and empty access masks	Synchronisation validation, read after write	correct
wrong-stage	`COPY` named as the stage of `vkCmdFillBuffer`, which runs in `CLEAR`	Synchronisation validation, read after write	correct
layout-mismatch	A copy from an image left in `GENERAL`, declared `TRANSFER_SRC_OPTIMAL`	Core validation, at recording	correct
war-reuse	A buffer uploaded again while the last dispatch may still read it	Synchronisation validation, write after read	wrong
host-coherency	Mapped memory used without flushes or invalidations	Nothing on this device	correct
semaphore-stage	A semaphore wait at the compute stage before a copy reads	Synchronisation validation, at submission	correct

Table 3.7.1. Six classic bugs on the M2 Pro

Five of the six gave the right answer. Only the reused buffer produced wrong results, because its upload raced a dispatch that was still reading. Without the validation layer, five of these programs would have passed every test on this machine.

Reading a report

A synchronisation validation report names the command that found the hazard, the kind of hazard, the resource, and the earlier access it conflicts with, and then states the stages and accesses that a barrier must connect. Object names given with `VK_EXT_debug_utils`, which `vkf` sets whenever it is given a name, label the handles with words such as `[a]`. The first bug's report is typical:

An execution dependency without a memory dependency, reported

```
$ ./build/bin/u3_bugs --bug missing-visibility
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
bug missing-visibility: an execution dependency with no memory dependency
[vulkan error] vkCmdDispatch(): READ_AFTER_WRITE hazard detected. Shader stage
    VK_SHADER_STAGE_COMPUTE_BIT reads VkBuffer 0x...[a], which was previously written by
    another vkCmdDispatch command.
The buffer is referenced by descriptor binding 0 (VK_DESCRIPTOR_TYPE_STORAGE_BUFFER) from
    descriptor set 0 (VkDescriptorSet 0x...), VkPipeline 0x...[transform.comp].
No sufficient synchronization is present to ensure that a read
    (VK_ACCESS_2_SHADER_STORAGE_READ_BIT) at VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT does not
    conflict with a prior write (VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT) at the same stage.
  synchronisation validation: SYNC-HAZARD-READ-AFTER-WRITE
  results: correct on this device
error: --bug missing-visibility detected by synchronisation validation
vkf: the validation layers reported 1 error(s)
```

The read at the compute stage was ordered after the write, but the write was never made visible to it. The report's last sentence gives the fix, a source access of `SHADER_STORAGE_WRITE` and a destination access of `SHADER_STORAGE_READ` at the stages already named.

The first bug, and its fix — code/u3/bugs/main.cpp

```
const VkMemoryBarrier2 visibility{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = bug ? 0 : VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = bug ? 0 : VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
};
barrier(cmd, visibility);
```

The catalogue

The second bug names the wrong stage. `vkCmdFillBuffer` and `vkCmdUpdateBuffer` are *clear* commands and run in `VK_PIPELINE_STAGE_2_CLEAR_BIT`, so a barrier whose source is `COPY` does not wait for them. The layer reports a read after write at the dispatch, naming the stage the write really happened in:

The stage of a fill is CLEAR — code/u3/bugs/main.cpp

```
vkCmdFillBuffer(cmd, e.a, 0, VK_WHOLE_SIZE, 7);
const VkMemoryBarrier2 fillDone{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = bug ? VK_PIPELINE_STAGE_2_COPY_BIT : VK_PIPELINE_STAGE_2_CLEAR_BIT,
    .srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
};
barrier(cmd, fillDone);
```

The third bug leaves an image in the wrong layout. The copy declares `TRANSFER_SRC_OPTIMAL`, but no barrier ever moved the image there. This is a core validation error, reported when the copy is recorded, because the layer tracks each subresource's layout through the command buffer:

A transition the copy depends on code/u3/bugs/main.cpp

```
const VkImageMemoryBarrier2 toCopy{
    .sType = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .srcAccessMask = VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
    .dstAccessMask = VK_ACCESS_2_TRANSFER_READ_BIT,
    .oldLayout = VK_IMAGE_LAYOUT_GENERAL,
    .newLayout = bug ? VK_IMAGE_LAYOUT_GENERAL : VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL,
    .srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED,
    .image = image,
    .subresourceRange = vkf::colorRange(),
};
const VkDependencyInfo dependency{
    .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
    .imageMemoryBarrierCount = 1,
    .pImageMemoryBarriers = &toCopy,
};
vkCmdPipelineBarrier2(cmd, &dependency);
const VkBufferImageCopy region{
    .imageSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1},
    .imageExtent = {Side, Side, 1},
};
vkCmdCopyImageToBuffer(cmd, image, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, e.readback, 1,
                       &region);
```

The layout the copy declares does not match the layout the image is in

```
$ ./build/bin/u3_bugs --bug layout-mismatch
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
bug layout-mismatch: a copy from an image left in GENERAL
[vulkan error] vkCmdCopyImageToBuffer(): srcImage Cannot use VkImage 0x...[image] (layer 0,
    mip 0) with specific layout VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL that doesn't match the
    previous known layout VK_IMAGE_LAYOUT_GENERAL.
The Vulkan spec states: srcImageLayout must specify the layout of the image subresources of
    srcImage specified in pRegions at the time this command is executed on a VkDevice
  core validation: VUID-vkCmdCopyImageToBuffer-srcImageLayout-00189
  results: correct on this device
error: --bug layout-mismatch detected by core validation
vkf: the validation layers reported 1 error(s)
```

The fourth bug reuses a buffer. A loop uploads data into one buffer, dispatches a kernel that reads it, and uploads the next batch into the same buffer. Without a barrier, the next upload may overwrite the buffer while the last dispatch still reads it, a write after read. An execution dependency is enough to prevent it, as the report says, because the hazard involves no earlier write to make visible. The fix's destination access also covers the next copy's write after the previous copy's.

Waiting for the last reader before uploading again — code/u3/bugs/main.cpp

```
if (k > 0 && !bug) {
    // Waits for the last dispatch's reads, and covers the last copy's write.
    const VkMemoryBarrier2 reuse{
        .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
        .srcStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
        .srcAccessMask = VK_ACCESS_2_NONE,
        .dstStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
        .dstAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
    };
    barrier(cmd, reuse);
}
const VkBufferCopy region{.srcOffset = k * Count * 4, .size = Count * 4};
vkCmdCopyBuffer(cmd, staging, e.a, 1, &region);
const VkMemoryBarrier2 copied{
    .sType = VK_STRUCTURE_TYPE_MEMORY_BARRIER_2,
    .srcStageMask = VK_PIPELINE_STAGE_2_COPY_BIT,
    .srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT,
    .dstStageMask = VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
    .dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT,
};
barrier(cmd, copied);
u3::dispatch(cmd, e.transform, sets[k], CountPush{Count}, Count / 256);
```

The only bug of the six whose results went wrong on the M2 Pro

```
$ ./build/bin/u3_bugs --bug war-reuse
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
bug war-reuse: a buffer overwritten while the last dispatch may read it
[vulkan error] vkCmdCopyBuffer(): WRITE_AFTER_READ hazard detected. vkCmdCopyBuffer writes
    to VkBuffer 0x...[a], which was previously read by vkCmdDispatch.
No sufficient synchronization is present to ensure that a write
    (VK_ACCESS_2_TRANSFER_WRITE_BIT) at VK_PIPELINE_STAGE_2_COPY_BIT does not conflict with
    a prior read (VK_ACCESS_2_SHADER_STORAGE_READ_BIT) at
    VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT.
Hint: An execution dependency is sufficient to prevent this hazard.
Buffer copy region 0: {
  offset = 0,
  size = 262144
}

...
  synchronisation validation: SYNC-HAZARD-WRITE-AFTER-READ
  results: wrong
error: --bug war-reuse detected by synchronisation validation
vkf: the validation layers reported 3 error(s)
```

The fifth bug forgets that mapped memory may not be coherent. The host's writes to memory without `HOST_COHERENT` reach the device only after `vkFlushMappedMemoryRanges`, and the device's writes reach the host's view only after `vkInvalidateMappedMemoryRanges`, in addition to the barriers and fences that order them.

Flushing host writes before the device reads them — code/u3/bugs/main.cpp

```
// Host writes to non-coherent memory reach the device only after a flush.
if (!bug) {
    const VkMappedMemoryRange written{
        .sType = VK_STRUCTURE_TYPE_MAPPED_MEMORY_RANGE,
        .memory = input.memory,
        .offset = 0,
        .size = VK_WHOLE_SIZE,
    };
    VKF_CHECK(vkFlushMappedMemoryRanges(e.ctx.device(), 1, &written));
}
```

Invalidating before the host reads the device's writes — code/u3/bugs/main.cpp

```
// Without an invalidate, the host may read stale lines from its own caches.
if (!bug) {
    const VkMappedMemoryRange result{
        .sType = VK_STRUCTURE_TYPE_MAPPED_MEMORY_RANGE,
        .memory = output.memory,
        .offset = 0,
        .size = VK_WHOLE_SIZE,
    };
    VKF_CHECK(vkInvalidateMappedMemoryRanges(e.ctx.device(), 1, &result));
}
```

Every host-visible memory type on the M2 Pro is coherent, so on this machine the bug cannot be observed, and the layer does not track host accesses at all. A program written here and run on a GPU with non-coherent memory types, which several mobile and some desktop GPUs have, can fail without any report.

The sixth bug waits too late in the pipeline. A semaphore wait's stage mask says which stages of the waiting submission must wait. Here the first command after the wait is a copy, but the wait names only the compute stage, so the copy may read before the signalling submission has written. The report comes at `vkQueueSubmit2`, where synchronisation validation checks the dependencies between submissions:

A wait stage that must cover the first reader — code/u3/bugs/main.cpp

```
const auto written = vkf::createSemaphore(e.ctx);
const auto fence = vkf::createFence(e.ctx);
const vkf::SemaphoreSubmit signal{written, 0, VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT};
const vkf::SemaphoreSubmit wait{
    written, 0,
    bug ? VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT : VK_PIPELINE_STAGE_2_COPY_BIT};
vkf::submit(e.ctx.mainQueue(), std::span(&produce, 1), {}, std::span(&signal, 1));
vkf::submit(e.ctx.mainQueue(), std::span(&consume, 1), std::span(&wait, 1), {}, fence);
vkf::waitFence(e.ctx, fence);
```

A hazard across submissions, found at submission

```
$ ./build/bin/u3_bugs --bug semaphore-stage
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
bug semaphore-stage: a semaphore wait at COMPUTE_SHADER before a copy reads
[vulkan error] vkQueueSubmit2(): pSubmits[0].pCommandBufferInfos[0] READ_AFTER_WRITE hazard
    detected. vkCmdCopyBuffer (from VkCommandBuffer 0x... submitted on the current VkQueue
    0x...[main queue]) reads VkBuffer 0x...[a], which was previously written by
    vkCmdDispatch (from VkCommandBuffer 0x... submitted on VkQueue 0x...[main queue]).
The current synchronization allows
    VK_ACCESS_2_UNIFORM_READ_BIT|VK_ACCESS_2_SHADER_READ_BIT|
    VK_ACCESS_2_ACCELERATION_STRUCTURE_READ_BIT_KHR|VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT|
    VK_ACCESS_2_DESCRIPTOR_BUFFER_READ_BIT_EXT|VK_ACCESS_2_SAMPLER_HEAP_READ_BIT_EXT|
    VK_ACCESS_2_RESOURCE_HEAP_READ_BIT_EXT
    accesses at VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT, but to prevent this hazard, it must
    allow VK_ACCESS_2_TRANSFER_READ_BIT accesses at VK_PIPELINE_STAGE_2_COPY_BIT.
Buffer copy region 0: {
  offset = 0,
  size = 262144
}

  synchronisation validation: SYNC-HAZARD-READ-AFTER-WRITE
  results: correct on this device
error: --bug semaphore-stage detected by synchronisation validation
vkf: the validation layers reported 1 error(s)
```

Blind spots

The course's programs found these limits of synchronisation validation in version 1.4.363 of the layer:

- **Host accesses.** The layer does not track the host, so missing host barriers, flushes and invalidations go unreported (Chapter 3.1's `host-device` case and the fifth bug above).

- **Accesses through device addresses.** Shaders that reach buffers through `GL_EXT_buffer_reference` are invisible to it (Chapter 2.6).
- **Atomics.** The layer counts atomic operations as reads. A missing barrier after a pass whose only writes are atomics goes unreported, and a barrier that orders such a pass but makes nothing visible is accepted.
- **Aliased images.** Hazards between buffers that share memory are reported. Hazards between images that share memory with each other or with buffers are not, which matters for the aliasing of Chapter 3.8.
- **Ownership transfers.** An acquire without a release is a core error at submission. A release without an acquire, followed by a use, is not reported.
- **Races within a dispatch.** Invocations of one dispatch that race through shared or global memory are outside its model (Chapter 2.2).

It can also be wrong the other way. With the shader-access heuristic that `vkf` enables, the layer assumes that a shader accesses the whole of every buffer range bound to it. Two dispatches that write disjoint slices of one buffer bound whole are reported as a write after write. Binding each dispatch's slice, with a separate descriptor or a dynamic offset, as Chapter 3.3's cost experiment does, removes the report without a barrier.

A method for what validation misses

When results are wrong and the layer is silent, or a bug appears on one GPU only:

1. **Synchronise everything.** Replace every barrier with a *big hammer*, a barrier whose masks are `ALL_COMMANDS` and `MEMORY_READ` with `MEMORY_WRITE`, and wait for the device to go idle after each submission. If the bug disappears, it is probably a synchronisation bug. Restore the real barriers one at a time to find it.
2. **Widen the race window.** A missing dependency shows only if the consumer runs before the producer finishes. Make the producer slower, with more work or a spin loop, and run the program many times.
3. **Change the hardware.** Run on GPUs from more than one vendor, or on a software implementation such as Mesa's lavapipe, which schedules work differently.
4. **Inspect the data.** Capture the work with RenderDoc or Xcode and inspect the buffers between commands, or print from shaders with debug printf (Chapter 1.6).
5. **Keep the layer in the tests.** Every test should run with synchronisation validation and fail on any message, as every program in this course does.

> KEY IDEA
> A synchronisation bug usually gives correct results where it was written. Run synchronisation validation on every test, read each report for the two accesses and the masks that would connect them, and know the layer's blind spots, testing what it cannot see by widening race windows and changing hardware.

> COMING FROM OPENGL
> OpenGL's version of these bugs is the missing `glMemoryBarrier` after a shader writes, which no layer reports and which a driver that waits conservatively hides. The rest of the catalogue has no OpenGL counterpart, because the driver chose stages, layouts and waits itself, and kept mapped buffers coherent unless the program asked otherwise.

Reading

- The validation layer's documentation, "Synchronization Validation", in the Vulkan-ValidationLayers repository, including its list of known limitations.
- LunarG, "Guide to Vulkan Synchronization Validation", a white paper on the layer's design and use.
- The Khronos wiki page "Synchronization Examples", as a reference for the fixes.

Labs

LAB 3.7.1 Fix the bugs from their reports — 2 hours

Goal. Fix each bug from its report alone.

Code. `code/u3/bugs`

1. Run `u3_bugs --bug NAME` for each of the six bugs, with validation on.
2. For each, write the fix from the report alone, without reading the program's fixed version, then compare.

DONE WHEN

- [ ] Your six fixes pass with validation silent.
- [ ] Where your fix differs from the program's, you can say whether both are correct, and which is cheaper.

EVIDENCE FOR THE NOTEBOOK

- The six reports and your fixes.

LAB 3.7.2 An atomic blind spot — 1.5 hours

Goal. See a hazard the layer cannot report.

Code. `code/u3/bugs`

1. Write a pass that only increments counters with `atomicAdd`, followed by a pass that reads the counters, with no barrier between them.
2. Run with validation on, then widen the window by giving the first pass much more work, and run many times.

DONE WHEN

- [ ] Validation is silent, and you have either wrong results or an argument, from how your GPU runs the two passes, for why none appeared.
- [ ] You have added the barrier, and can name its masks.

EVIDENCE FOR THE NOTEBOOK

- The program, the outputs and the explanation.

LAB 3.7.3 From report to barrier — 1 hour

Goal. Translate reports into masks quickly.

Code. `code/u3/hazards`

1. Run three of Chapter 3.1's cases with `--bug`, and from each report alone write the barrier that fixes it.
2. Check each against the case's own barrier.

DONE WHEN

- [ ] All three of your barriers are correct.
- [ ] You can point to the part of each report that gave you each mask.

EVIDENCE FOR THE NOTEBOOK

- The reports, annotated.

LAB 3.7.4 The bugs on a second device — 1 hour

Goal. See the same bugs on different hardware.

Code. `code/u3/bugs`

1. Run every `--bug` variant on a second device: another GPU, or Mesa's lavapipe on Linux.
2. Compare which bugs produce wrong results there.

DONE WHEN

- [ ] You have a table of both devices' results.
- [ ] You can explain each difference from how the two devices schedule work.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

Problem set

1. Why did five of the six bugs produce correct results on the M2 Pro? What would make them fail there?
2. The semaphore-stage hazard is reported at `vkQueueSubmit2`, not while the command buffer is recorded. Why?
3. Two dispatches write disjoint halves of one buffer, bound whole, and the layer reports a write after write. Is it a real hazard? How would you silence the report without adding a barrier?
4. Write the big hammer's barrier. Why is a bug that disappears under it evidence of a synchronisation bug, and not proof?
5. A program that is silent under synchronisation validation fails on another vendor's GPU. List the blind spots you would examine first, and how you would test each.

COMPLETION CHECKLIST

- [ ] I can read a synchronisation validation report and write the barrier it asks for.
- [ ] I can recognise the six classic bugs and fix each.
- [ ] I know what synchronisation validation cannot see, and how to test it otherwise.
- [ ] I can hunt a synchronisation bug that the tools miss, systematically.
- [ ] Lab 3.7.1–3.7.4 done-when criteria all hold, with evidence filed.

CHAPTER 3.8

Render graphs

TIME	7 hours
YOU BUILD	A render graph library of about 900 lines, in which passes declare the resources they read and write. Compilation culls passes whose results nothing uses, places transient resources in one allocation with aliasing, derives every barrier and layout transition, and splits work across queues with timeline semaphores; execution records each pass with its barriers and times it. A demonstration graph derives exactly the barriers Chapter 3.3 wrote by hand, saves a fifth of its memory by aliasing, and runs with async compute.
NEEDS	Chapters 3.1 to 3.7.

WHY IT MATTERS

Every barrier in this unit so far was placed by hand. That works for a pipeline of seven passes written by one person. In a frame with dozens of passes, changed by many people, adding, removing or reordering a pass invalidates barriers elsewhere without any error, and hand-placed barriers become the most fragile code in the renderer. In a *render graph*, each pass declares the resources it reads and writes, and how, and the graph derives everything else: which passes matter, where temporary resources live, every barrier and layout transition, and how work splits across queues. Production engines organise their frames this way, and this chapter builds one small enough to read in an evening.

SKIP TEST

1. A pass reads a buffer that the pass before it wrote, and a pass between them also read it. What does the new pass's barrier wait for?
2. What does it take for a pass to be removed from a frame, and why must a readback be marked so that it never is?
3. Two transient images never live at the same time and share memory. What must the barrier before the second image's first use contain?
4. A pass moves to another queue. What replaces the barriers that connected it to its neighbours?

If all four are easy, read the sections on deriving barriers and on async compute, and do Labs 3.8.3 and 3.8.4.

Core ideas

Declaring a frame

A pass is a function that records commands, together with a list of the resources it uses. Each use states the pipeline stages that access the resource, the kinds of access, and, for an image, the layout the pass needs it in:

One use of one resource — code/u3/rendergraph/rendergraph.hpp

```
// How a pass uses a resource: the stages and accesses, and for an image the layout it needs.
struct Use {
    VkPipelineStageFlags2 stages = VK_PIPELINE_STAGE_2_NONE;
    VkAccessFlags2 access = VK_ACCESS_2_NONE;
    VkImageLayout layout = VK_IMAGE_LAYOUT_UNDEFINED;
};
```

Most uses are one of a few kinds, which Chapter 3.3's catalogue already listed. The library names them:

The common uses, with their stages, accesses and layouts — code/u3/rendergraph/rendergraph.hpp

```
inline constexpr Use ComputeRead{VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                                 VK_ACCESS_2_SHADER_STORAGE_READ_BIT, VK_IMAGE_LAYOUT_GENERAL};
inline constexpr Use ComputeWrite{VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT,
                                  VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT,
                                  VK_IMAGE_LAYOUT_GENERAL};
inline constexpr Use IndirectRead{VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT,
                                  VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT};
inline constexpr Use CopySource{VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_READ_BIT,
                                VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL};
inline constexpr Use CopyDestination{VK_PIPELINE_STAGE_2_COPY_BIT,
                                     VK_ACCESS_2_TRANSFER_WRITE_BIT,
                                     VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL};
inline constexpr Use ClearDestination{VK_PIPELINE_STAGE_2_CLEAR_BIT,
                                      VK_ACCESS_2_TRANSFER_WRITE_BIT,
                                      VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL};
inline constexpr Use HostRead{VK_PIPELINE_STAGE_2_HOST_BIT, VK_ACCESS_2_HOST_READ_BIT};
inline constexpr Use VertexInput{VK_PIPELINE_STAGE_2_VERTEX_ATTRIBUTE_INPUT_BIT,
                                 VK_ACCESS_2_VERTEX_ATTRIBUTE_READ_BIT};
inline constexpr Use IndexInput{VK_PIPELINE_STAGE_2_INDEX_INPUT_BIT,
                                VK_ACCESS_2_INDEX_READ_BIT};
inline constexpr Use FragmentSampled{VK_PIPELINE_STAGE_2_FRAGMENT_SHADER_BIT,
                                     VK_ACCESS_2_SHADER_SAMPLED_READ_BIT,
                                     VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL};
inline constexpr Use ColorAttachment{
    VK_PIPELINE_STAGE_2_COLOR_ATTACHMENT_OUTPUT_BIT,
    VK_ACCESS_2_COLOR_ATTACHMENT_READ_BIT | VK_ACCESS_2_COLOR_ATTACHMENT_WRITE_BIT,
    VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL};
inline constexpr Use DepthAttachment{
    VK_PIPELINE_STAGE_2_EARLY_FRAGMENT_TESTS_BIT | VK_PIPELINE_STAGE_2_LATE_FRAGMENT_TESTS_BIT,
    VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_READ_BIT |
        VK_ACCESS_2_DEPTH_STENCIL_ATTACHMENT_WRITE_BIT,
    VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL};
// Nothing in the queue waits for presentation; vkQueuePresentKHR waits on a semaphore instead.
inline constexpr Use Present{VK_PIPELINE_STAGE_2_NONE, VK_ACCESS_2_NONE,
                             VK_IMAGE_LAYOUT_PRESENT_SRC_KHR};
```

A pass adds uses with `read`, when it needs the resource's current contents, and `write`, when it produces new ones. A pass that updates a resource in place calls both. A pass whose effects are outside the graph, such as one that lets the host read results, is marked as a *side effect*, and a pass can ask for the async compute queue:

Declaring a pass — code/u3/rendergraph/rendergraph.hpp

```
class PassBuilder {
public:
    // The pass needs the resource's current contents.
    PassBuilder& read(Resource resource, Use use);
    // The pass produces new contents; also call read() when it reads the old ones.
    PassBuilder& write(Resource resource, Use use);
    // Keeps the pass even when no kept pass reads what it writes: readbacks, presentation.
    PassBuilder& sideEffect();
    // Runs on the async compute queue when the context has one, otherwise on the main queue.
    PassBuilder& queue(Queue queue);

private:
    friend class Graph;
    PassBuilder(Graph& graph, uint32_t pass) : graph_(graph), pass_(pass) {}
    Graph& graph_;
    uint32_t pass_;
};
```

Resources are either imported, made by the program, with the use they had before the graph runs, or *transient*: created by the graph for its own passes, and free to share memory with others.

Imported and transient resources — code/u3/rendergraph/rendergraph.hpp

```
// A resource made outside the graph; `previous` is its last use before the graph runs.
Resource importBuffer(std::string name, VkBuffer buffer, Use previous = {});
Resource importImage(std::string name, VkImage image, VkImageView view,
                     VkImageSubresourceRange range, Use previous = {});
// Made by compile(); transients whose lifetimes do not overlap may share memory.
Resource createBuffer(std::string name, VkDeviceSize size, VkBufferUsageFlags usage);
Resource createImage(std::string name, const vkf::ImageDesc& desc);
```

The demonstration declares Chapter 3.3's seven-pass pipeline as a graph, with three additions: a pass that clears the statistics, a pass that copies the blurred field for debugging, which nothing reads, and a pass that stands for the host reading the results.

Seven transients and one imported buffer — code/u3/rendergraph/main.cpp

```
const uint32_t side = options.side;
const VkDeviceSize bytes = VkDeviceSize(side) * side * 4;
const auto grid = [side](VkImageUsageFlags usage) {
    return vkf::ImageDesc{
        .format = VK_FORMAT_R32_UINT, .width = side, .height = side, .usage = usage};
};
constexpr VkBufferUsageFlags Storage = VK_BUFFER_USAGE_STORAGE_BUFFER_BIT;
constexpr VkBufferUsageFlags CopyBoth =
    VK_BUFFER_USAGE_TRANSFER_SRC_BIT | VK_BUFFER_USAGE_TRANSFER_DST_BIT;
const Ids r{
    .field = graph.createImage("field", grid(VK_IMAGE_USAGE_STORAGE_BIT)),
    .blurred =
        graph.createBuffer("blurred", bytes, Storage | VK_BUFFER_USAGE_TRANSFER_SRC_BIT),
    .edges = graph.createBuffer("edges", bytes, Storage),
    .mask = graph.createImage(
        "mask", grid(VK_IMAGE_USAGE_STORAGE_BIT | VK_IMAGE_USAGE_TRANSFER_SRC_BIT)),
    .stats = graph.createBuffer("stats", sizeof(demo::Stats), Storage | CopyBoth),
    .list = graph.createBuffer("list", bytes, Storage),
    .command = graph.createBuffer("command", sizeof(VkDispatchIndirectCommand),
                                  Storage | VK_BUFFER_USAGE_INDIRECT_BUFFER_BIT),
    .results = graph.importBuffer("results", results),
};
const rg::Resource scratch =
    graph.createBuffer("scratch", bytes, VK_BUFFER_USAGE_TRANSFER_DST_BIT);
```

The first five passes — code/u3/rendergraph/main.cpp

```
graph.addPass("clear-stats", [&p](VkCommandBuffer cmd) { p->clearStats(cmd); })
    .write(r.stats, rg::ClearDestination);
graph.addPass("generate", [&p](VkCommandBuffer cmd) { p->generate(cmd); })
    .write(r.field, rg::ComputeWrite);
graph.addPass("blur", [&p](VkCommandBuffer cmd) { p->blur(cmd); })
    .read(r.field, rg::ComputeRead)
    .write(r.blurred, rg::ComputeWrite);
graph.addPass("edges", [&p](VkCommandBuffer cmd) { p->edges(cmd); })
    .read(r.field, rg::ComputeRead)
    .write(r.edges, rg::ComputeWrite)
    .queue(options.async ? rg::Queue::Compute : rg::Queue::Main);
const auto debugCopy = [&graph, r, scratch, bytes](VkCommandBuffer cmd) {
    const VkBufferCopy region{.size = bytes};
    vkCmdCopyBuffer(cmd, graph.buffer(r.blurred), graph.buffer(scratch), 1, &region);
};
graph.addPass("debug-copy", debugCopy)
    .read(r.blurred, rg::CopySource)
    .write(scratch, rg::CopyDestination);
```

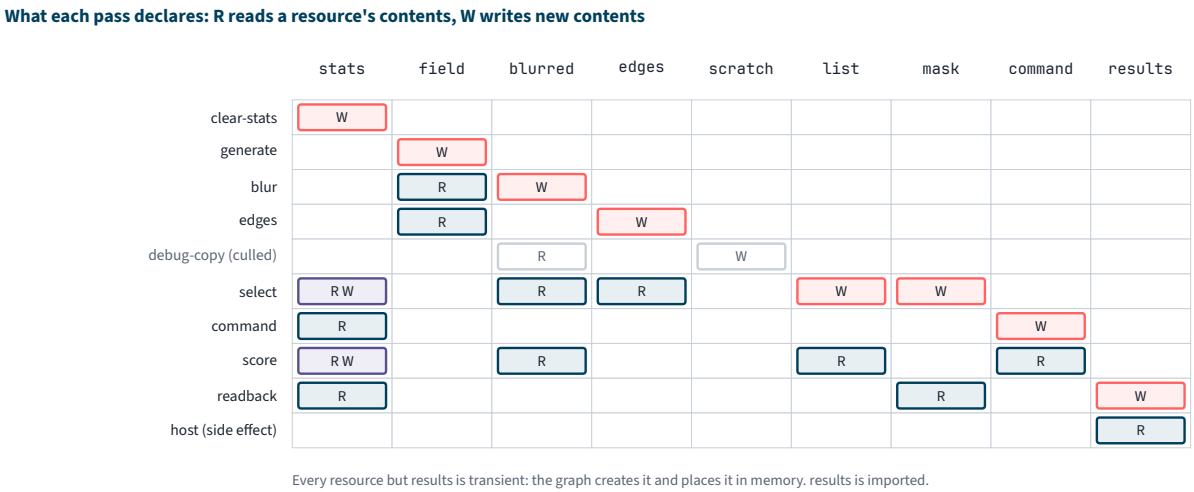


Figure 3.8.1. The demonstration's declarations. Each row is a pass, in the order declared; each column a resource.

The last five passes — code/u3/rendergraph/main.cpp

```
graph.addPass("select", [&p](VkCommandBuffer cmd) { p→select(cmd); })
    .read(r.blurred, rg::ComputeRead)
    .read(r.edges, rg::ComputeRead)
    .read(r.stats, rg::ComputeRead)
    .write(r.stats, rg::ComputeWrite)
    .write(r.list, rg::ComputeWrite)
    .write(r.mask, rg::ComputeWrite);
graph.addPass("command", [&p](VkCommandBuffer cmd) { p→command(cmd); })
    .read(r.stats, rg::ComputeRead)
    .write(r.command, rg::ComputeWrite);
graph.addPass("score", [&p](VkCommandBuffer cmd) { p→score(cmd); })
    .read(r.command, rg::IndirectRead)
    .read(r.list, rg::ComputeRead)
    .read(r.blurred, rg::ComputeRead)
    .read(r.stats, rg::ComputeRead)
    .write(r.stats, rg::ComputeWrite);
graph.addPass("readback", [&p](VkCommandBuffer cmd) { p→readback(cmd); })
    .read(r.stats, rg::CopySource)
    .read(r.mask, rg::CopySource)
    .write(r.results, rg::CopyDestination);
graph.addPass("host").read(r.results, rg::HostRead).sideEffect();
```

Culling

Compilation first decides which passes to keep. A pass with a side effect is kept, and so is any pass that wrote something a kept pass reads. A single walk through the passes in reverse order finds them all, because every dependency points backwards. The debug copy writes only `scratch`, which nothing reads, so it is culled. A kept pass that reads a transient nothing has written is an error, caught here:

Keeping the passes that lead to a side effect — code/u3/rendergraph/rendergraph.cpp

```
// A pass survives if it has a side effect or a surviving pass reads something it wrote.
void Graph::cull() {
    const auto writes = [&](const Pass& p, Resource r) {
        return std::any_of(p.accesses.begin(), p.accesses.end(),
                           [&](const Access& a) { return a.resource == r && a.writes; });
    };
    for (Pass& pass : passes_) pass.kept = pass.sideEffect;
    for (size_t i = passes_.size(); i-- > 0;) {
        if (!passes_[i].kept) continue;
        for (const Access& a : passes_[i].accesses) {
            if (!a.reads) continue;
            size_t j = i;
            while (j > 0 && !writes(passes_[j - 1], a.resource)) --j;
            if (j > 0) {
                passes_[j - 1].kept = true;
```

```
            } else if (resources_[a.resource].transient) {
                throw std::runtime_error("pass '" + passes_[i].name + "' reads '" +
                                         resources_[a.resource].name +
                                         "' before any pass writes it");
            }
        }
    }
    for (uint32_t i = 0; i < passes_.size(); ++i) {
        if (passes_[i].kept) order_.push_back(i);
    }
}
```

A pass that reads a transient before anything writes it

```
$ ./build/bin/u3_rendergraph --bug read-unwritten
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
by hand: 9 passes, 7 vkCmdPipelineBarrier2 calls, results match the CPU
graph without aliasing: the same barriers as by hand at every pass
error: pass 'orphan' reads 'unwritten' before any pass writes it
```

Placing transients

Each kept transient lives from its first user to its last. Two transients whose lifetimes do not overlap can occupy the same memory. The graph creates every transient, gathers their memory requirements, and places them in order of first use, each at the lowest offset where it overlaps no transient that is alive at the same time. Placement respects each resource's alignment, and keeps a live buffer and a live image apart by `bufferImageGranularity`, so that they never share a page of the size the device requires.

First fit, in order of first use — code/u3/rendergraph/rendergraph.cpp

```
// A live buffer and image must not share a page of bufferImageGranularity bytes.
const VkDeviceSize granularity = ctx_.properties().core.limits.bufferImageGranularity;
for (size_t i = 0; i < transients.size(); ++i) {
    ResourceInfo& r = resources_[transients[i]];
    for (bool moved = true; moved;) {
        moved = false;
        for (size_t j = 0; j < i; ++j) {
            const ResourceInfo& other = resources_[transients[j]];
            if (aliasMemory && (other.last < r.first || r.last < other.first)) continue;
            const VkDeviceSize page = other.isImage != r.isImage ? granularity : 1;
            const VkDeviceSize start = other.offset / page * page;
            const VkDeviceSize end = vkf::alignUp(other.offset + other.memory.size, page);
            if (r.offset < end && start < vkf::alignUp(r.offset + r.memory.size, page)) {
                r.offset = vkf::alignUp(end, r.memory.alignment);
                moved = true;
            }
        }
    }
    aliasedBytes_ = std::max(aliasedBytes_, r.offset + r.memory.size);
}
```

In the demonstration, `mask` is first written after `field` is last read, and takes its memory. `command` takes the memory of `edges` the same way (Figure 3.8.2). The graph's transients need 4 MiB instead of 5.

A resource that takes over memory starts with undefined contents, but the hardware may still be using that memory for its previous occupant. Its first use therefore needs a barrier whose source is the previous occupant's last accesses, and, for an image, a transition from `UNDEFINED`. The graph treats the newcomer as inheriting the old resource's accesses, which produces exactly that. Synchronisation validation cannot check this hand-over for images (Chapter 3.7), so the graph must be right by construction.

Deriving barriers

The graph then simulates the frame, pass by pass, keeping for each resource what has happened to it so far on each queue: the stages and accesses of its last write, the stages that have read it since, which of those reads were already made to see the write, and its layout.

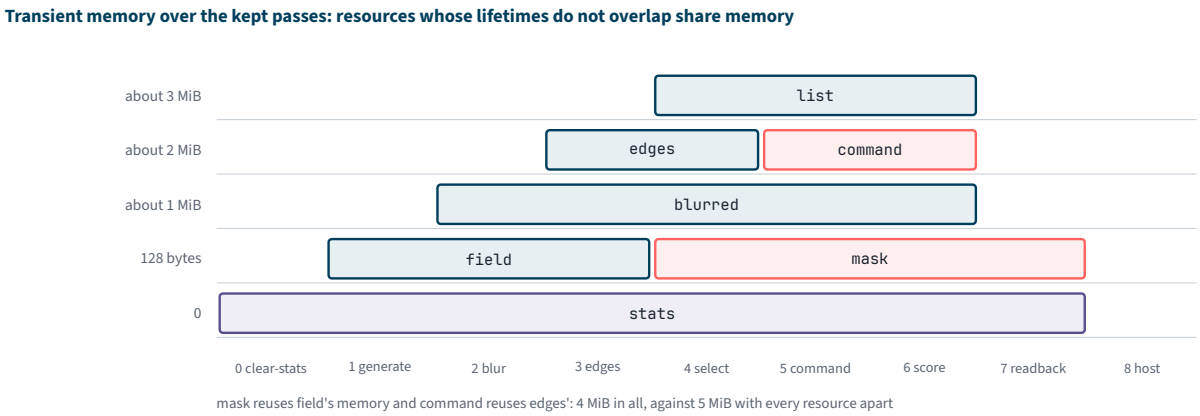


Figure 3.8.2. Where the transients live, and when. Two resources in one row share memory.

What the graph knows about a resource between passes — code/u3/rendergraph/rendergraph.cpp

```
// A resource's memory between two passes, as compilation sees it on each queue.
struct Graph::Sim {
    struct OnQueue {
        VkPipelineStageFlags2 writeStages = 0, readStages = 0, waitedStages = 0;
        VkAccessFlags2 writeAccess = 0;
        int writeBatch = -1, readBatch = -1;
        std::vector<Use> seenBy;

        bool sees(const Use& use) const {
            return std::any_of(seenBy.begin(), seenBy.end(), [&](const Use& s) {
                return (use.stages & ~s.stages) == 0 && (use.access & ~s.access) == 0;
            });
        }
    };
    OnQueue queue[2];
    VkImageLayout layout = VK_IMAGE_LAYOUT_UNDEFINED;

    // Takes on the uses of a resource whose memory this one now occupies.
    void absorb(const Sim& old) {
        for (int q = 0; q < 2; ++q) {
            queue[q].writeStages |= old.queue[q].writeStages;
            queue[q].writeAccess |= old.queue[q].writeAccess;
            queue[q].readStages |= old.queue[q].readStages;
            queue[q].waitedStages |= old.queue[q].waitedStages;
            queue[q].writeBatch = std::max(queue[q].writeBatch, old.queue[q].writeBatch);
            queue[q].readBatch = std::max(queue[q].readBatch, old.queue[q].readBatch);
        }
    }
};
```

At each use, three rules decide the barrier:

- A use that *modifies* the resource, by writing it or by changing an image's layout, must wait for the last write, a write after write, and for every read since, a write after read. The last write's accesses become the barrier's source accesses. The reads contribute stages only, since an execution dependency is enough to protect a read.
- A use that only reads must wait for the last write and have it made visible to its stages and accesses, unless an earlier read has already been given exactly that. A read after a read needs nothing.
- A dependency on work in another queue's batch becomes a semaphore wait instead.

The barrier one use needs — code/u3/rendergraph/rendergraph.cpp

```
const Use& use = a.use;
const VkAccessFlags2 writes = use.access & WriteAccess;
const bool transition = info.isImage && use.layout != s.layout;
const bool modifies = writes != 0 || transition;
Dependency d{.resources = {a.resource},
             .dstStages = use.stages,
```

```
            .dstAccess = use.access,
            .oldLayout = s.layout,
            .newLayout = use.layout,
            .transition = transition};
Sim::OnQueue& mine = s.queue[q];
bool waited = false;
for (uint32_t x = 0; x < 2; ++x) {
    const Sim::OnQueue& prior = s.queue[x];
    if (prior.writeStages ≠ 0 && (modifies || !mine.sees(use))) {
        if (x == q) {
            d.srcStages ⊨ prior.writeStages;
            d.srcAccess ⊨ prior.writeAccess;
        } else if (mine.waitedStages ≠ 0) {
            d.srcStages ⊨ mine.waitedStages;
        } else if (prior.writeBatch ≥ 0) {
            wait(batch, prior.writeBatch, use.stages);
            waited = true;
        }
    }
    if (modifies && prior.readStages ≠ 0) {
        if (x == q) {
            d.srcStages ⊨ prior.readStages;
        } else if (prior.readBatch ≥ 0) {
            wait(batch, prior.readBatch, use.stages);
            waited = true;
        }
    }
}
// A layout transition after a semaphore wait must chain from the waiting stages.
if (waited && transition) d.srcStages ⊨ use.stages;
if (d.srcStages ≠ 0 || transition) needs.push_back(d);
```

After each use, the simulation records what the use did:

Recording a use — code/u3/rendergraph/rendergraph.cpp

```
if (modifies) {
    s.queue[0] = s.queue[1] = {};
    s.layout = use.layout;
    mine.writeStages = use.stages;
    mine.writeAccess = writes;
    mine.writeBatch = batch;
    if (writes == 0) {
        // Only a layout transition: its writes are visible to this read already.
        mine.seenBy.push_back(use);
        mine.readStages = use.stages;
        mine.readBatch = batch;
    }
    continue;
}
if (d.srcStages ≠ 0 || waited) mine.seenBy.push_back(use);
if (waited) mine.waitedStages ⊨ use.stages;
mine.readStages ⊨ use.stages;
mine.readBatch = batch;
```

Finally the dependencies before each pass are grouped into one `vkCmdPipelineBarrier2` call: one global memory barrier for each pair of stage masks, merging their accesses, and one image barrier for each layout transition. The graph uses global memory barriers even for buffers. Buffer barriers name a range, which drivers rarely exploit, and global barriers keep the plan short.

One barrier per pair of stage masks, one per transition code/u3/rendergraph/rendergraph.cpp

```
// One memory barrier per pair of stage masks; each layout transition its own.
for (Dependency& d : needs) {
    auto same = std::find_if(plan[k].begin(), plan[k].end(), [&](const Dependency& g) {
        return !d.transition && !g.transition && g.srcStages == d.srcStages &&
               g.dstStages == d.dstStages;
    });
    if (same == plan[k].end()) {
        plan[k].push_back(d);
        continue;
    }
    same->srcAccess |= d.srcAccess;
    same->dstAccess |= d.dstAccess;
    same->resources.push_back(d.resources[0]);
}
```

The result is the plan the graph prints. Each pass lists its barriers, with the resources they protect:

The demonstration's plan, with aliasing

```
$ ./build/bin/u3_rendergraph
...
compiled: 10 passes, 9 kept, culled: debug-copy
transient memory: 4096 KiB with aliasing, 5120 KiB without
  field    image   1048576 bytes at 128      passes 1-3
  blurred  buffer  1048576 bytes at 1048832  passes 2-6
  edges    buffer  1048576 bytes at 2097408  passes 3-4
  mask     image   1048576 bytes at 128      passes 4-7, reuses field
  stats    buffer       16 bytes at 0        passes 0-7
  list     buffer  1048576 bytes at 3145984  passes 4-6
  command  buffer       12 bytes at 2097408  passes 5-6, reuses edges
pass 0 clear-stats
pass 1 generate
  image  field UNDEFINED -> GENERAL
    src NONE             NONE
    dst COMPUTE_SHADER   SHADER_STORAGE_WRITE
pass 2 blur
  memory field
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ
pass 3 edges
pass 4 select
  memory blurred, edges
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ
  memory stats
    src CLEAR            TRANSFER_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ|SHADER_STORAGE_WRITE
  image  mask (memory of field) UNDEFINED -> GENERAL
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_WRITE
pass 5 command
  memory stats, command (memory of edges)
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ|SHADER_STORAGE_WRITE
pass 6 score
  memory command
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst DRAW_INDIRECT    INDIRECT_COMMAND_READ
  memory list, stats
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ|SHADER_STORAGE_WRITE
pass 7 readback
  memory stats
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COPY             TRANSFER_READ
  image  mask GENERAL -> TRANSFER_SRC_OPTIMAL
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COPY             TRANSFER_READ
```

```
pass 8 host
  memory results
    src COPY              TRANSFER_WRITE
    dst HOST              HOST_READ
...
```

Before `select`, for example, `blurred` and `edges` share one barrier, since both were written by compute shaders and are read by one. `stats` needs its own, because its last write was the clear, in another stage. `mask` arrives in `field`'s memory with a transition from `UNDEFINED`, after compute-shader work that `field` saw.

To check the derivation, the demonstration also runs Chapter 3.3's pipeline with barriers written by hand, then compiles the graph without aliasing and compares the two, barrier by barrier. They match at every pass:

The same pipeline with its barriers written by hand — code/u3/rendergraph/byhand.cpp

```
r.before();
p.clearStats(cmd);
r.before({}, {image(t.field, VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_GENERAL,
                    VK_PIPELINE_STAGE_2_NONE, VK_ACCESS_2_NONE, Compute, Write)});
p.generate(cmd);
r.before({memory(Compute, Write, Compute, Read)});
p.blur(cmd);
r.before();
p.edges(cmd);
r.before({memory(Compute, Write, Compute, Read),
          memory(VK_PIPELINE_STAGE_2_CLEAR_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT, Compute,
                 Read | Write)},
         {image(t.mask, VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_GENERAL,
                VK_PIPELINE_STAGE_2_NONE, VK_ACCESS_2_NONE, Compute, Write)});
p.select(cmd);
r.before({memory(Compute, Write, Compute, Read)});
p.command(cmd);
r.before({memory(Compute, Write, VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT,
                 VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT),
          memory(Compute, Write, Compute, Read | Write)});
p.score(cmd);
r.before(
    {memory(Compute, Write, VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_READ_BIT)},
    {image(t.mask, VK_IMAGE_LAYOUT_GENERAL, VK_IMAGE_LAYOUT_TRANSFER_SRC_OPTIMAL, Compute,
           Write, VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_READ_BIT)});
p.readback(cmd);
r.before({memory(VK_PIPELINE_STAGE_2_COPY_BIT, VK_ACCESS_2_TRANSFER_WRITE_BIT,
                 VK_PIPELINE_STAGE_2_HOST_BIT, VK_ACCESS_2_HOST_READ_BIT)});
```

Executing

A compiled graph is executed by recording its passes in order, each preceded by its barriers. The graph also brackets each pass with timestamps and with a debug label, so that captures and profilers show the passes by name:

Recording a batch of passes — code/u3/rendergraph/rendergraph.cpp

```
void Graph::recordBatch(const Batch& batch, VkCommandBuffer cmd) {
    const uint32_t family =
        batch.queue == Queue::Main ? ctx_.mainQueue().family : ctx_.computeQueue().family;
    const bool timed = ctx_.properties().queueFamilies[family].timestampValidBits > 0;
    for (uint32_t k : batch.passes) {
        const Barriers barriers = vulkanBarriers(k);
        if (!barriers.memory.empty() || !barriers.images.empty()) {
            const VkDependencyInfo dependency{
                .sType = VK_STRUCTURE_TYPE_DEPENDENCY_INFO,
                .memoryBarrierCount = uint32_t(barriers.memory.size()),
                .pMemoryBarriers = barriers.memory.data(),
                .imageMemoryBarrierCount = uint32_t(barriers.images.size()),
                .pImageMemoryBarriers = barriers.images.data(),
            };
            vkCmdPipelineBarrier2(cmd, &dependency);
```

```
        }
        const Pass& pass = passes_[order_[k]];
        if (timed) {
            vkCmdResetQueryPool(cmd, timestamps_, 2 * k, 2);
            vkCmdWriteTimestamp2(cmd, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, timestamps_,
                                 2 * k);
        }
        ctx_.beginLabel(cmd, pass.name.c_str());
        if (pass.record) pass.record(cmd);
        ctx_.endLabel(cmd);
        if (timed) {
            vkCmdWriteTimestamp2(cmd, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT, timestamps_,
                                 2 * k + 1);
        }
    }
}
```

Fifty executions on one queue, with validation off

```
$ VKF_VALIDATION=0 ./build/bin/u3_rendergraph --repeats 50
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
by hand: 9 passes, 7 vkCmdPipelineBarrier2 calls, results match the CPU
graph without aliasing: the same barriers as by hand at every pass
...
GPU ms per pass, median of 50: clear-stats 0.010, generate 0.016, blur 0.064, edges 0.004,
  select 0.028, command 0.005, score 0.016, readback 0.021, host 0.005
CPU ms per execution (record, submit, wait), median of 50: 0.928
PASS: 51 executions match the CPU (24174 cells selected)
```

A graph is compiled once and executed many times. One graph per frame slot, as in Chapter 3.6, gives each frame in flight its own command buffers and transients.

Async compute

A pass that asks for the compute queue runs there when the device has a separate one. The graph then cuts the kept passes into *batches*, runs of consecutive passes on one queue, cutting after each pass that another queue waits for and before each pass that waits:

Cutting the passes into batches code/u3/rendergraph/rendergraph.cpp

```
// Cutting after waited-on passes and before waiting ones lets independent work overlap.
void Graph::formBatches() {
    std::vector<int> alone(order_.size());
    std::iota(alone.begin(), alone.end(), 0);
    std::vector<Batch> single(order_.size());
    derive(alone, single);
    std::vector<bool> starts(order_.size()), ends(order_.size());
    for (uint32_t k = 0; k < order_.size(); ++k) {
        for (const auto& wait : single[k].waits) {
            ends[wait.first] = true;
            starts[k] = true;
        }
    }
    std::vector<int> batchOf(order_.size());
    int open[2] = {-1, -1};
    for (uint32_t k = 0; k < order_.size(); ++k) {
        const int q = int(queueOf(k));
        if (open[q] < 0 || starts[k]) {
            batches_.push_back({.queue = Queue(q)});
            open[q] = int(batches_.size() - 1);
        }
        batchOf[k] = open[q];
        batches_[open[q]].passes.push_back(k);
        if (ends[k]) open[q] = -1;
    }
    plan_ = derive(batchOf, batches_);
    // The last batch signals the caller's semaphores, so it waits for the other queue too.
    Batch& last = batches_.back();
```

```
    for (int i = int(batches_.size()) - 2; i >= 0; --i) {
        if (batches_[i].queue == last.queue) continue;
        if (std::none_of(last.waits.begin(), last.waits.end(),
                         [&](const auto& w) { return w.first == i; })) {
            last.waits.push_back({i, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT});
        }
        break;
    }
}
```

Each queue has a timeline semaphore. A batch signals its queue's timeline when it finishes, and waits for the values of the batches it depends on, at the stages that need them. Transients used by both queues are created with `CONCURRENT` sharing, so that no ownership transfers are needed. The last batch also waits for the other queue's last batch, so that the caller's own semaphores and fence cover the whole graph.

Submitting the batches with timeline semaphores — code/u3/rendergraph/rendergraph.cpp

```
void Graph::submit(std::span<const vkf::SemaphoreSubmit> waits,
                   std::span<const vkf::SemaphoreSubmit> signals, VkFence fence) {
    for (auto& pool : pools_) {
        if (pool) VKF_CHECK(vkResetCommandPool(ctx_.device(), pool, 0));
    }
    bool mainWaited = false;
    for (Batch& batch : batches_) {
        vkf::beginCommands(batch.cmd);
        recordBatch(batch, batch.cmd);
        vkf::endCommands(batch.cmd);
        const int q = int(batch.queue);
        batch.value = ++submitted_[q];
        std::vector<vkf::SemaphoreSubmit> waitFor, signal;
        for (const auto& [producer, stages] : batch.waits) {
            const Batch& from = batches_[producer];
            waitFor.push_back({timelines_[int(from.queue)], from.value, stages});
        }
        if (batch.queue == Queue::Main && !mainWaited) {
            waitFor.insert(waitFor.end(), waits.begin(), waits.end());
            mainWaited = true;
        }
        signal.push_back({timelines_[q], batch.value, VK_PIPELINE_STAGE_2_ALL_COMMANDS_BIT});
        const bool last = &batch == &batches_.back();
        if (last) signal.insert(signal.end(), signals.begin(), signals.end());
        const VkQueue queue = q == 0 ? ctx_.mainQueue().queue : ctx_.computeQueue().queue;
        vkf::submit(queue, std::span(&batch.cmd, 1), waitFor, signal,
                    last ? fence : VK_NULL_HANDLE);
    }
}
```

With `--async`, the demonstration moves `edges` to the compute queue:

Four batches across two queues, with validation on

```
$ ./build/bin/u3_rendergraph --async
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
...
batch 0 on the main queue: clear-stats generate
batch 1 on the main queue: blur
batch 2 on the compute queue: edges; waits for batch 0 at COMPUTE_SHADER
batch 3 on the main queue: select command score readback host; waits for batch 2 at
    COMPUTE_SHADER
...
pass 4 select
  memory blurred
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ
  memory stats
    src CLEAR            TRANSFER_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_READ|SHADER_STORAGE_WRITE
```

```
  image  mask (memory of field) UNDEFINED → GENERAL
    src COMPUTE_SHADER   SHADER_STORAGE_WRITE
    dst COMPUTE_SHADER   SHADER_STORAGE_WRITE
...
GPU ms per pass, median of 10: clear-stats 0.013, generate 0.011, blur 0.050, edges 0.118,
  select 0.109, command 0.005, score 0.017, readback 0.025, host 0.004
CPU ms per execution (record, submit, wait), median of 10: 1.304
PASS: 11 executions match the CPU (24174 cells selected)
```

The graph is correct with async compute, and slower: 1.21 ms per execution against 0.93. Its passes take only about 0.25 ms of GPU time in all, so the extra submissions and semaphore waits cost more than the overlap saves. As Chapter 3.5 measured, a second queue pays when the work it takes is large and the first queue leaves hardware idle. Chapter 4.7 puts a larger workload, a particle simulation, on the compute queue.

Graphics passes

The same rules cover rendering. Attachments are images whose uses name the attachment stages and layouts, and a draw that clears its attachments writes them without reading:

A graph with colour and depth attachments, a copy and a read on the host

```
$ ./build/bin/u3_rendergraph --graphics
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation on, synchronisation validation on
compiled: 3 passes, 3 kept, culled: none
transient memory: 32 KiB with aliasing, 32 KiB without
  colour   image     16512 bytes at 0        passes 0-1
  depth    image     16512 bytes at 16512    passes 0-0
pass 0 draw
  image  colour UNDEFINED → COLOR_ATTACHMENT_OPTIMAL
    src NONE             NONE
    dst COLOR_ATTACHMENT_OUTPUT COLOR_ATTACHMENT_READ|COLOR_ATTACHMENT_WRITE
  image  depth UNDEFINED → DEPTH_STENCIL_ATTACHMENT_OPTIMAL
    src NONE             NONE
    dst EARLY_FRAGMENT_TESTS|LATE_FRAGMENT_TESTS
        DEPTH_STENCIL_ATTACHMENT_READ|DEPTH_STENCIL_ATTACHMENT_WRITE
pass 1 copy
  image  colour COLOR_ATTACHMENT_OPTIMAL → TRANSFER_SRC_OPTIMAL
    src COLOR_ATTACHMENT_OUTPUT COLOR_ATTACHMENT_WRITE
    dst COPY             TRANSFER_READ
pass 2 host
  memory pixels
    src COPY             TRANSFER_WRITE
    dst HOST             HOST_READ
PASS: the colour attachment holds the clear colour
```

Render graphs in production

Frostbite's *frame graph* introduced the idea to a wide audience in 2017, and Unreal Engine's Render Dependency Graph and many other engines follow it. Production graphs do more than this one. They reorder passes to overlap independent work, merge passes into render passes with subpasses for tile-based GPUs, use events where independent work can run between a producer and its consumer, give attachments lazily allocated memory on tilers, track subresources such as single mip levels, and keep resources from one frame for the next, such as the history buffers of temporal effects. The design is the same: the program declares its uses and the graph derives the barriers.

> **KEY IDEA**
> Declare what each pass reads and writes, and let the graph derive the passes that matter, where transients live, every barrier and layout transition, and how work splits across queues. A graph that derives the barriers a careful programmer would write makes adding a pass safe.

> **COMING FROM OPENGL**
> An OpenGL driver does part of a render graph's work. It sees each pass's resources as the program binds them, and derives waits and cache flushes at run time, with no knowledge of what comes next. A Vulkan render graph does that work ahead of time, once per frame structure, and sees the whole frame, which lets it cull, alias and schedule as no driver can.

Reading

- Yuriy O'Donnell, "FrameGraph: Extensible Rendering Architecture in Frostbite", GDC 2017.
- Hans-Kristian Arntzen, "Render graphs and Vulkan: a deep dive", 2017, on a graph for Vulkan in the Granite engine.
- Epic Games, "Render Dependency Graph", in the Unreal Engine documentation.
- Graham Wihlidal, "Halcyon Architecture: Director's Cut", SEED, 2018.

Labs

LAB 3.8.1 Add a pass — 1.5 hours

Goal. Extend a graph, and check what it derives.

Code. `code/u3/rendergraph`

1. Add a pass after `select` that counts the cells of `mask` in a new transient buffer, and make `readback` copy that count out too.
2. Write, by hand, the barriers your pass needs, then compare them with the plan the graph prints.

DONE WHEN

- [ ] The count is correct, and validation is silent.
- [ ] Your barriers match the graph's, or you can explain every difference.

EVIDENCE FOR THE NOTEBOOK

- The changed declarations, your barriers and the plan.

LAB 3.8.2 Memory aliasing — 1.5 hours

Goal. Make aliasing visible and check its barriers.

Code. `code/u3/rendergraph`

1. Run with and without `--no-alias`, and compare the placements.
2. Add a transient image that is used only after `select`, and find where the graph places it and what its first barrier contains.

DONE WHEN

- [ ] Your image's results are correct, and validation is silent.
- [ ] You can explain each reuse in the plan, and why its first barrier names the stages it does.

EVIDENCE FOR THE NOTEBOOK

- The two plans and the explanation.

LAB 3.8.3 From one frame to the next — 2 hours

Goal. Carry a resource between executions.

Code. `code/u3/rendergraph`

1. Import a buffer that accumulates the statistics of every execution, read and written by a new pass, with its previous use set to that pass's use.
2. Execute the graph ten times and check the accumulated totals.

DONE WHEN

- [ ] The totals are correct after ten executions, and validation is silent.
- [ ] You can explain the barrier the graph derives before the first use in each execution.

EVIDENCE FOR THE NOTEBOOK

- The changes and the output.

LAB 3.8.4 Schedule for overlap — 1.5 hours

Goal. Judge which work belongs on the compute queue.

Code. `code/u3/rendergraph`

1. Move each of `blur`, `edges` and `select` to the compute queue in turn, and print each plan's batches.
2. Time each configuration with validation off, at `--side 512` and `--side 2048`.

DONE WHEN

- [ ] You have a table of batches and times.
- [ ] You can explain which placements add semaphores without overlap, and why the larger size changes the picture.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

Problem set

1. Before `select`, `blurred` and `edges` share one barrier, but `stats` has its own. Why?
2. The barrier before `mask`'s first use has a source access of `SHADER_STORAGE_WRITE`, although nothing has written `mask`. Why?
3. Why is the debug copy culled, and what is the smallest change to the declarations that would keep it?
4. The graph uses global memory barriers for buffers. What would buffer barriers add, and when would they be worth it?
5. Describe a frame in which reordering two passes would let more work overlap, and what a graph must know to reorder them safely.

COMPLETION CHECKLIST

- [ ] I can declare a frame as passes that read and write resources, with the right stages, accesses and layouts.
- [ ] I can cull passes, place transients with aliasing, and say what a resource's first use in shared memory needs.
- [ ] I can derive the barriers between passes from their declarations, as the graph does.
- [ ] I can split a graph across queues with timeline semaphores, and judge when that pays.
- [ ] Lab 3.8.1–3.8.4 done-when criteria all hold, with evidence filed.

UNIT 3

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and repository, and you can do each review task below without notes. Date each line as you finish it. If someone who writes GPU code for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
3.1	Why synchronisation is your job	☐	☐	
3.2	The dependency model	☐	☐	
3.3	Pipeline barriers in practice	☐	☐	
3.4	Fences, semaphores and events	☐	☐	
3.5	Queues and async compute	☐	☐	
3.6	Frames in flight	☐	☐	
3.7	Finding synchronisation bugs	☐	☐	
3.8	Render graphs	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook and code.

- ☐ For any producer and consumer in Chapter 3.3's catalogue, write the barrier's masks and layouts from memory, and justify each with the dependency model.
- ☐ Draw Chapter 3.6's frame loop with three frames in flight, marking every wait and signal of the timeline semaphore and what each frame slot owns.
- ☐ Explain a queue family ownership transfer, release and acquire, and when an upload to another family needs none.
- ☐ Take a synchronisation validation report from your notebook and explain, line by line, the hazard and its fix.
- ☐ Derive by hand the barriers before two passes of the render graph's demonstration, and check them against its plan.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____